

Sehat AI: A Healthcare Assistant for Every Pakistani



Authors

Muhammad Abdul Rehman, Abdul Qadir Abbasi, Muhammad

Tahawar Ali

22-EE-40, 22-EE-44, 22-EE-72

Supervisor

Dr. Furqan Shoukat

Associate Professor

DEPARTMENT OF ELECTRICAL ENGINEERING
FACULTY OF ELECTRONICS & ELECTRICAL ENGINEERING
UNIVERSITY OF ENGINEERING AND TECHNOLOGY
TAXILA

June 2026

Sehat AI: A Healthcare Assistant for Every Pakistani



Authors

Muhammad Abdul Rehman, Abdul Qadir Abbasi, Muhammad

Tahawar Ali

22-EE-40, 22-EE-44, 22-EE-72

Supervisor

Dr. Furqan Shoukat

Associate Professor

DEPARTMENT OF ELECTRICAL ENGINEERING
FACULTY OF ELECTRONICS & ELECTRICAL ENGINEERING
UNIVERSITY OF ENGINEERING AND TECHNOLOGY
TAXILA

June 2026

*This thesis submitted in partial fulfillment of the requirements for the degree of
B.Sc. Electrical Engineering*

Supervisor:

Dr. Furqan Shoukat

Associate Professor

Electrical Engineering Department

Undertaking

We certify that research work titled “Sehat AI: A healthcare assistant for every Pakistani” is my own work. The work has not been presented elsewhere for assessment. Where material has been used from other sources it has been properly acknowledged / referred.

Signature of Student(s)

Name of Student(s)

Muhammad Abdul Rehman

Abdul Qadir Abbasi

Muhammad Tahawar Ali

Registration Number(s)

22-EE-40, 22-EE-44, 22-EE-72

External Examiner Signature: _____

Thesis Supervisor Signature: _____

DEDICATION

We dedicate this thesis to my parents, whose love, support, and encouragement have helped me throughout my studies. Their sacrifices, prayers, and belief in us made this achievement possible. I am deeply grateful for everything they have done for me.

TABLE OF CONTENTS

DEDICATION	ii
Table of Contents	iii
Abstract.....	1
Acknowledgments.....	2
Chapter 1: Introduction	3
1.1 General.....	3
1.2 Problem Statement.....	4
1.3 Objectives:	5
1.4 Scope of the Work	5
1.4.1 Dataset Curation and Preprocessing.....	6
1.4.2 Framework and Model Development.....	6
1.4.3 Software Architecture and EMR Creation	7
1.4.4 Evaluation, Safety, and Deployment.....	7
Chapter 2: Literature Review	9
2.1 Introduction and Theoretical Framework	9
2.2 Medical Large Language Models and Clinical Adaptation	9
2.3 Parameter-Efficient Fine-Tuning (PEFT):.....	10
2.3.1 Low-Rank Adaptation (LoRA) Fundamentals.....	10
2.3.2 Adaptive Budget Allocation (AdaLoRA)	10
2.3.3 SFT vs. LoRA vs. In-Context Learning (ICL)	11
2.3.4 LoRA in Cross-Domain Applications	11
2.4 Model Quantization: Compressing LLMs for Edge Deployment	11
2.4.1 The Challenge of Emergent Outliers at Scale	12
2.4.2 SmoothQuant: Migrating Difficulty to Weights	12
2.4.3 Advanced Low-Bit Algorithms: GPTQ, HQQ, and QuIP	12
2.4.4 Quantization-Aware Training (QAT).....	13
2.5 Hardware Acceleration and Edge Deployment Constraints	14
2.6 Evaluation of Accuracy-Performance Trade-Offs.....	14
2.7 Potential Avenues and Bottlenecks	15
2.8 Conclusive Remarks	16
Chapter 3 System Design and Implementation.....	17
3.1 Introduction to the Sehat AI Architecture	17
3.2 Dataset Curation and Preprocessing Pipeline	19
3.3 Foundation Model Selection.....	19
3.4 Parameter-Efficient Fine-Tuning (PEFT) utilizing LoRA	20
3.4.1 Mathematical Formulation of LoRA.....	20
3.4.2 Hyperparameter Configuration	20
3.5 Post-Training Quantization and Edge Optimization	21
3.5.1 The GGUF Format and 4-Bit Compression	21
3.5.2 Compression Results	21

3.6 Software Architecture and Electronic Medical Record (EMR) Integration	22
3.6.1 Application Backend and Frontend.....	22
3.6.2 Core EMR Functionalities.....	24
3.6.3 Interoperability via FHIR R4	24
3.8 System Deployment and Hardware Configuration.....	24
3.8.1 The Ollama Adapter	24
3.9 Block Diagram.....	25
Chapter 4 Results and Case Studies.....	26
4.1 Introduction	26
4.2 Model Compression and Hardware Efficiency Results.....	26
4.2.1 Parameter-Efficient Fine-Tuning (PEFT) Optimization	26
4.2.2 Quantization Footprint and Inference Speed.....	27
4.3 Quantitative Benchmark Evaluation.....	28
4.3.1 Benchmark Dataset Performance	28
4.3.2 Analysis of Quantitative Results	29
4.4 Qualitative Clinical Case Studies	29
4.4.1 Case Study 1: Primary Care Triage (Gastroenteritis).....	30
4.4.2 Case Study 2: Dermatological Assessment	30
4.4.3 Case Study 3: Clinical Decision Support for Professionals	31
4.5 Software Architecture and EMR Integration Evaluation	31
4.5.1 Local Deployment and Latency Testing	32
4.5.2 Data Interoperability and FHIR Compliance	32
4.5.3 Automated Report Generation	32
4.6 Discussion on Hardware Acceleration and Future Avenues	33
Chapter 5 Conclusion	33
5.1 Summary of Achievements	34
5.2 Key Findings and Discoveries	34
5.3 Societal Impact and Future Outlook	35
Chapter 6 Suggestions for future work	36
6.1 Suggestions for Future Work.....	36
References	38
Abbreviations.....	40
Annixture A & Code Implementation Details	43
A.1: Overview and Purpose	44
A.1.1 Supported Hardware & Runtime	44
A.2 Environment Setup and Dependency Installation	46
A.2.1 Key Library Versions and Their Roles	47
A.2.2 Why Version Pinning Matters	48
A.3 Model Initialisation — FastLanguageModel.....	49
A.3.1 Parameter Explanation — from_pretrained()	49
A.3.2 Unsloth Speed Optimisations	50
A.4 LoRA Adapter Configuration	51

A.4.1 LoRA Hyperparameter Analysis	52
A.4.2 Target Modules Selection	53
A.5 Data Preparation Pipeline.....	55
F.5.1 Chat Template Configuration.....	55
A.5.2 Dataset Loading	56
A.5.3 Conversation Format Conversion	57
A.5.4 Chat Template Application.....	57
A.6 Training Configuration — SFTTrainer.....	59
A.6.1 Hyperparameter Reference Table	60
A.6.2 Response-Only Training Mask	61
A.7 GPU Memory Profiling and Training Execution	63
A.7.1 Expected Memory Breakdown	64
A.7.2 Interpreting Training Statistics	64
A.8 Inference — Generating Clinical Responses.....	65
A.8.1 Inference Parameter Reference	66
A.8.2 Thinking vs. Non-Thinking Inference	67
A.9 Saving and Exporting the Fine-Tuned Model	68
A.9.1 Saving LoRA Adapters Only.....	68
A.9.2 Merged Weight Export (Float16 and 4-bit).....	69
A.9.3 GGUF Export for Ollama / llama.cpp	70
A.10 Reloading Saved Adapters for Inference.....	72
A.10.1 Deployment Architecture Summary	72
A.11 End-to-End Pipeline Summary.....	74

ABSTRACT

Pakistan's health care system is under a severe human resource crunch, as there are only about 4 doctors for every 6000 people in rural areas, which is causing people to seek assistance from unqualified individuals. To fill this critical gap, this research has created the "Sehat AI" Electronic Medical Record (EMR) and healthcare management system that is designed to be 100% off-line and privacy conscious, with an embedded medical Large Language Model (LLM). We used LoRA to fine-tune the foundational model, Qwen3-4B-Instruct, on the Medical-R1-Distill dataset, and added Chain-of-Thought reasoning to generate safe and evidence-based clinical decision support. The framework leveraged aggressive post-training quantization to 4-bit format for low-cost deployment without relying on the cloud in resource-poor clinics.

The main findings of this work illustrate the clinical performance and applicability of edge-based clinical AI. The quantized MedQwen3-4B-Q4 model successfully passed the tough medical tests with an overall accuracy of 48.6% on them, including MedQA (60.6%) and MedMCQA (55.4%). Most importantly, the quantization process managed to reduce the model footprint to a mere 2.6 GB, 85.6% of the original (nearly seven times) smaller than the standard 8 billion parameter medical models while preserving the integrity of the clinical reasoning of the model. Moreover, system integration testing proved this lightweight architecture is fully functional locally without the need for an Internet connection, on standard computer hardware. The offline system was able to handle patient data that was FHIR R4 compliant, and provided real-time, context-aware differential diagnosis and treatment offers. These findings demonstrate a viable and scalable solution to enhance diagnostic precision and alleviate burden with high-performing, privacy-conscious AI, in the absence of centralized infrastructure.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to my supervisor Dr. Furqan Shoukat for his valuable guidance, constant help and encouragement during the course of this research work. I would like to express my gratitude to Dr. Laiq ur Rehman for giving me the knowledge which helped me in this job and also to teach me Machine Learning. Special thanks to Dr. Junaid Mir for excellent teaching of Python programming and Image processing, giving me the required technical foundation to complete this work. Last, but not least, I would like to thank all those who helped me either directly or indirectly in the process of completing my thesis.

CHAPTER 1: INTRODUCTION

1.1 General

The start of Artificial Intelligence (AI) and healthcare is one of the significant technological advancements of the 21st century. AI is quickly coming into existence and transforming important industries such as education, agriculture, and particularly medicine. Large Language Models (LLMs), like ChatGPT, DeepSeek, and Qwen, have consistently excelled at comprehending, combining, and creating human language, leading the charge in this revolution. In the health care field, these developments are making their way into clinical processes at an unprecedented pace, providing sophisticated tools to help increase the accuracy of diagnosis, deliver complex drug recommendations and dramatically transform hospital management systems.

The applications where AI is most clearly visible are the ones that drive the reduction of the load on hospitals and the improvement of the quality of the care provided – for example, being able to deliver healthcare remotely and providing highly personalized treatment plans. The digital transformation is already being leveraged by developed countries. As of now, it's estimated that more than 377 health systems and providers have trialled or implemented over 70 different generative AI tools from 49 different vendors in the United States. As a result, most U.S. doctors now are currently using AI in their day-to-day practice. Globally, more than 48% of clinicians across 109 countries are actively using AI for their everyday work, and in China, more than 300 hospitals are deploying local AI models such as DeepSeek to help medical professionals.

Even with all these huge global advances, developing countries and the Low- and Middle-Income Countries (LMICs) such as Pakistan are far behind in the race of AI. Such regions are typically neglected in current AI research and application in healthcare, where access to high-end AI computational infrastructure, medical data and native language support is very limited. The lack of digital health ecosystems adoption has real and significant population health impacts. There is a great need for scalable, low-bandwidth infrastructure that can easily integrate into clinical settings and other low-

resource contexts to bridge this growing technology and healthcare divide. To address this gap, the proposed solution, “Sehat AI”, aims to create a digital health ecosystem that is accessible, accurate, affordable, and can function entirely offline.

1.2 Problem Statement

Pakistan's healthcare system is today facing great and unsustainable stresses, with a significant and widening gap between the numbers of health workers and the needs of a rapidly increasing population. Official statistics of the Pakistan Medical and Dental Council (PMDC) show that there are around 339,017 registered doctors with a population of well over 256 million in Pakistan. The number of doctors per 1,000 people on paper is approximately 1:755, the recommended minimum in the WHO guidelines is 1:1,000.

But this statistic doesn't tell the true story of the health care workforce. An estimate of the number of doctors who are not actively practicing medicine was shown to be as many as 30,000 to 40,000 between the 2024 briefing submitted to the Senate Standing Committee. Besides this, the PMDC data reveals a serious gender imbalance in workforce, as almost 80% of the students enrolled in medical colleges in Pakistan are female, but more than 50% of them either do not start a profession or leave it within a few years of their graduation.

This "brain drain" and workforce attrition inevitably translates to a functional and real-world ratio of doctors to patients which is significantly compromised, especially in rural and underserved geographical regions. According to the statistics shared by ORIC, KMU, Peshawar in Khyber Pakhtunkhwa province there are only about 1 functional doctor per 6,000 people. This acute lack of skilled healthcare staff has induced the majority of the population to look for unqualified practitioners (quacks) to deal with their most basic primary health care. These unqualified people are not trained to give medical advice, which is often traditional, unverified and even harmful, and carries a serious risk for patients and leads to misdiagnoses. There are general-purpose Large Language Models (LLMs) that are very susceptible to “hallucination,” confidently creating incorrect information and presenting it as factual reality.

1.3 Objectives:

The main objective of this research is to create an efficient and effective AI healthcare companion and Electronic Medical Record (EMR) system designed for resource-constrained settings. This project has the following broad aims to directly address the issues outlined in the problem statement:

1. To create an AI healthcare management system with accurate medical information, symptom analysis, disease guidance and personalized treatment plans for use in low-resource environments, ready for clinical deployment.
2. Clinical Support for Profession: To enable physician and medical staff to have clinical decision support tools with AI assistance. This involves support for complicated diagnosis, drug interaction monitoring, medicine choice and evaluation of medical history, thus minimizing diagnostic mistakes and improving clinical effectiveness in overcrowded hospitals.
3. To enable the public to be empowered to give themselves health guidance and care, so that they can be familiar with their symptoms, have reliable first step health advice, and know the exact time to go and get medical attention. This early intervention will be made to prevent minor problems from becoming more major, which will save the unnecessary burden on the infrastructure of hospitals.
4. Medical Quackery Elimination: To aggressively reduce the dangers of the public to doctors and quacks and unqualified practitioners by giving millions of people a reliable and scientifically sound information source that they can get from their homes.
5. Accessible and Offline Infrastructure: To engineer a secure low bandwidth, cost-effective infrastructure of the Healthcare system which works 100% off the grid with Local AI integration (Ollama). This guarantees that there is no dependence on the cloud, no operational cost, and no patient data privacy questions.

1.4 Scope of the Work

This work is significantly multi-disciplinary, from advanced machine learning engineering, through dataset curation, full stack software development, to rigorous

clinical evaluation. The project is aimed at providing a full and deployable digital health ecosystem mapping to Sustainable Development Goals (SDG 3: Good Health and Well-being, SDG 9: Industry, Innovation, and Infrastructure, and SDG 10: Reduced Inequalities). The scope is then further broken down into the following distinct domains:

1.4.1 Dataset Curation and Preprocessing

The project uses high quality verifiable medical problems generated by Stanford University and University of Hong Kong, Shenzhen in order to make sure that the model has deep clinical reasoning and not just memorization. The sourcing of medical MCQs for the curation process starts with the collection of 192,000 medical MCQs from known exams (MedQA-USMLE and MedMCQA). Relying on a traditional multiple-choice exam to produce a correct answer can lead to guessing, so a multi-stage filtering pipeline is used. Simple questions, which can be answered by smaller models such as Gemma2-9B, LLaMA-3.1-8B and Qwen2.5-7B, are intentionally omitted. GPT-4o then is used to eliminate ambiguity and reformat the remaining complex questions into open-ended structures. Last but not least, complex “Chain of Thought” (CoT) reasoning traces are produced by the full-precision DeepSeek R1 model. Traces that do not culminate in correct answers are thrown out, favoring longer and more detailed reasoning paths. This intensive scope produces around 20,000 high-quality samples, totalling more than 40 million tokens, to be used for specialized Supervised Fine-Tuning (SFT).

1.4.2 Framework and Model Development

Sehat AI's core intelligence model is the Qwen3-4B-Instruct-2507 model. Training involves Parameter-Efficient Fine-Tuning (PEFT), which is done through Low-Rank Adaptation (LoRA). To successfully balance between the efficiency of a model and its ability to capture the complex medical domain, LoRA adds low-rank matrices (adapters) to the model's self-attention and feed-forward networks, drastically decreasing the mathematical complexity from $O(n^2)$ to $O(n)$ and enabling a deep adaptation of the model to the medical domain. Post-training quantization is a no-go for this model to run on standard computers in low-resource clinics. The model is compressed into a 4-bit GGUF format (Q4_K_XL via llama.cpp) and it successfully compressed the memory

footprint of the model to only 2.6 GB. This quantization reduces the model size by 85.6%, or by almost seven times smaller than standard 8-billion parameter models, with competitive clinical accuracy. Furthermore, a novel approach called Group Relative Policy Optimization (GRPO) is discussed to improve reward-based medical alignment and edge-device image understanding, alongside Vision-Language Models (Liquid AI's LFM2).

1.4.3 Software Architecture and EMR Creation

The engineering work entails the creation of a complete 100% private Electronic Medical Record (EMR) system. The software architecture is a modern one, consisting of HTML5, CSS3, and vanilla JavaScript for the presentation layer, and Python 3.12 Flask 3.0.0 for the backend. The application includes patient registration, visit tracking, medical history (allergies, medications, surgeries), vital signs tracking (BP, HR, BMI), and lab result management. The AI layer is synced through Ollama and offers a real-time streaming chat UI which runs offline and analyses the patient's local records to deliver context-aware differential diagnosis and drug interaction checks. Global healthcare data standards are also hard coded into the software, with a built-in FHIR R4 processor for standardizing all data import/export and automated medical report generation into word documents (.docx). Most importantly, all architecture is cloud API independent, meaning complete HIPAA compliance for data privacy and no cost of operation.

1.4.4 Evaluation, Safety, and Deployment

The system is tested very rigorously before it goes to deployment, both quantitatively and qualitatively. MedQA, MedMCQA, PubMedQA, MMLU-Pro Health, and GPQA are the rigorously tested datasets in the scope of benchmarking. The quantized MedQwen3-4B-Q4 has a good performance of 48.6% average on these benchmarks. In addition to automation metrics, the scope must also involve pilot testing by licensed local clinicians who must review outputs of the systems for medical relevance, cultural appropriateness, and ethical compliance in a human-in-the-loop environment. Alibaba's Qwen3Guard safety model is embedded as an architectural protective filter to ensure patient safety. This layer filters out and prevents any kind of made-up info, hazardous suggestions, or

unethical generation from reaching the end user in real-time text generation stream. Last but not least, the scope of deployment is local clinic hardware, with a minimum of 8GB of RAM, Windows/Linux.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction and Theoretical Framework

The application of Large Language Models (LLMs) in the medical field marks a revolutionary advancement in digital healthcare, with the promise of making medical information more accessible and delivering advanced clinical decision support. Their use on resource limited environments, like rural Pakistan, where the internet connection is only available on an occasional basis and specialized hardware is not available, however, presents important engineering and science challenges. The theoretical foundation of this research (Sehat AI project) is based on three cutting-edge domains of AI research: (1) Domain-Specific Adaptation (teaching general models medical expertise), (2) Parameter-Efficient Fine-Tuning (PEFT) (teaching medical expertise without massive computational overhead), and (3) Model Quantization (compressing the models to run offline on edge devices).

This literature review offers a critical analysis of 16 state-of-the-art research paper and technical framework published by prestigious societies and institutions. It assesses the state-of-the-art in the fields of LLM adaptation, quantization techniques, and hardware acceleration, outlining the theoretical frameworks and any identified bottlenecks, guiding the architecture decisions for the Sehat AI infrastructure.

2.2 Medical Large Language Models and Clinical Adaptation

General-purpose LLMs (like ChatGPT) have a very limited understanding of medicine. Zhang et al. argue that models that are trained using only general internet information do not make interactive, expert-level clinical diagnosis. When it comes to offering advice, the ChatGPT model is likely to generate some broad, high-level suggestions, listing several alternatives without requesting further clarifying questions—an essential aspect of an actual doctor-patient interaction. On the other hand, models that are trained with real clinical data alone are of poor quality because doctors' notes are often very short and informal or incoherent.

To address this, Zhang et al. suggested a model, named HuatuoGPT, which integrates a hybrid dataset that combines distilled data from ChatGPT and real-world data from doctors. The authors were able to use a Supervised Fine-Tuning (SFT) stage and Reinforcement Learning from AI Feedback (RLAIF) to achieve the fluent, patient-friendly style of ChatGPT, while retaining the rigorous, interactive and diagnostic nature of human physicians. It represents a critical theoretical foundation for Sehat AI: Effective Medical AI isn't just about pre-training data, it needs to be supervised by combining structured reasoning data with clinical truth.

2.3 Parameter-Efficient Fine-Tuning (PEFT):

However, since the foundation models are growing in size to billions of parameters, the Full Fine-Tuning (FFT) is impractical, as it demands a large number of GPUs. A lot of research has been dedicated to Parameter-Efficient Fine-Tuning (PEFT) to avoid this bottleneck. [2]

2.3.1 Low-Rank Adaptation (LoRA) Fundamentals

The original paper for Low-Rank Adaptation (LoRA) by Hu et al., transformed the field of model tuning. The authors have shown that the weight updates in the fine-tuning have an intrinsic low "rank". LoRA freezes the pre-trained weight matrices and adds small matrices, A and B, that are trainable in the individual layers of the Transformer. By reducing the number of trainable parameters by up to 10,000 times and the amount of GPU memory required by 3 times, it does not degrade from the performance of FFT, and even equals or exceeds the performance of it. Finally, Raschka explained that calculating the approximations of weight changes through such low rank matrices is a way for models to be able to learn in a specific domain (medical classification, for instance) without the catastrophic impacts of pre-training.

2.3.2 Adaptive Budget Allocation (AdaLoRA)

Standard LoRA allocates a similar amount of parameters to all weight matrices, but Zhang proposed AdaLoRA. They said the differences in the significance of various weight parameters are neglected, which results in suboptimal fine-tuning. AdaLoRA

dynamically allocates the parameter budget by parameterizing the incremental updates as singular value decomposition (SVD). AdaLoRA significantly shrinks the parameter footprint by discarding less significant updates, with only minor exact SVD calculations, whose benefits are clearly discernible in low-budget scenarios. [4]

2.3.3 SFT vs. LoRA vs. In-Context Learning (ICL)

In the past, Bohnet conducted a comparative study of adaptation methods in the case of data scarcity. The researchers discovered an important drawback in current Supervised Fine-Tuning (SFT): catastrophic forgetting. If a model is very overfitted on a small task, its general reasoning and teaching skills immediately get worse. Bohnet et al. did the weight update analysis, and they found that LoRA is a powerful regularizer because it confines the updates to a low rank subspace. The research found that while SFT could learn a particular skill more quickly, LoRA offers a better balance of learning a new medical/clinical skill along with little loss of knowledge from the base model. [2]

2.3.4 LoRA in Cross-Domain Applications

The versatility of LoRA is further supported by the fact that it was also used by Yu to re-score Large Language Models in Speech Recognition. The authors were able to achieve competitive performance by using only 0.08% of the pre-trained parameters of a BERT model, which reduced the training memory consumption by 32%. Yu et al. managed to successfully implement a correlation-based regularization loss to avoid the degradation in representation that can be observed in LoRA. This validates that using LoRA with the right regularization is a very powerful approach to taking a generic LLM and adapting it to a very specific domain, such as healthcare. [5]

2.4 Model Quantization: Compressing LLMs for Edge Deployment

Local deployment of a multi-billion parameter model to a resource-constrained clinic also needs to tackle the inference memory bottleneck. However, the local deployment of a multi-billion parameter model to a resource-constrained clinic also needs to tackle the inference memory bottleneck. The mathematical operation of quantizing continuous

weights and activations to lower bit discrete integers, such as INT8 or INT4, which greatly reduces memory usage. [6]

2.4.1 The Challenge of Emergent Outliers at Scale

In 2023, Dettmers et al. published a seminal paper describing the reasons why standard quantization methods for large scale LLMs are inadequate. Highly systematic, large-magnitude outlier features appear in the hidden states as models get larger and larger (more than 6.7 billion parameters). These outliers can be as large as 20 times the normal dimensions, and would cause standard 8-bit quantization to be useless due to the following reasons: they would expand the range, causing all of the values except for the outliers to be zero, and thereby destroying the perplexity and accuracy of the model. To overcome this, Dettmers et al. chose to decompose the problem into two parts: multiply 99.9% dimensions using 8-bit integer numbers (INT8); multiply the remaining ~0.1% dimensions using 16-bit floating-point numbers (FP16).

2.4.2 SmoothQuant: Migrating Difficulty to Weights

For the outlier problem, Xiao has implemented something different in SmoothQuant. Xiao et al. noted that activations are hard to decompose in a way that is efficient on the hardware accelerators, while activations have outliers and the model weights are relatively uniform and easy to quantize. SmoothQuant performs a mathematically equivalent per-channel scaling transformation, where the difficulty of the quantization is shifted to the weights, thereby migrating the quantization challenge from the activations to the weights. SmoothQuant can preprocess the activation outliers before quantization to enable hardware-friendly 8-bit weight/8-bit activation quantization with standard integer matrix multiplication kernels, with up to 1.56x speedup and half memory usage with no loss of accuracy. [7]

2.4.3 Advanced Low-Bit Algorithms: GPTQ, HQQ, and QuIP

Even more compression would need 4 bit and 2 bit weight quantization.

GPTQ: Frantar: GPTQ is a very influential post-training quantization (PTQ) algorithm by Frantar which compresses models to 4-bits. GPTQ employs one quantization layer at a time, and the inverse-Hessian matrix is used to determine the relative "importance" of each weight. If one weight has a quantization error, GPTQ intelligently spreads it across the rest of the weights in the same row, ensuring that the overall accuracy of the network doesn't change. [8]

Salient issues Badri and Shaji found with the calibration-based methods are calibration data bias and huge computational time, hence they propose a method called Half-Quadratic Quantization. They suggested to model the outliers by a hyper-Laplacian distribution, and to minimize the loss of outliers by a sparsity-promoting loss (l_p norm) in HQQ. Calibration-free, HQQ can quantumize even the largest models such as Llama-2-70B in minutes, a significant improvement over the 50x slower GPTQ method. [9]

Chee et al. addressed the super-tight budget of 2-bit quantization, known as QuIP (Quantization with Incoherence Processing). They use the idea that the "incoherent" (weights even in size) quantization method is optimal for the weight and Hessian matrices. QuIP projects the matrices on the product of randomly chosen orthogonal matrices (Kronecker products), which is a principled version of outlier suppression which ensures that they are incoherent. QuIP is the very first approach to a viable 2-bit LLM compression. [10]

2.4.4 Quantization-Aware Training (QAT)

Esser et al. investigated SiLQ (Simple Large Language Model Quantization-Aware Training) while the Post-Training Quantization (PTQ) method is quick. In QAT the quantization step is not applied after training the model, but rather QFPs are used to fine-tune the model. SiLQ showed that, through knowledge distillation and the straight-through estimator, a "quantized" model can significantly outperform the best performing PTQ approaches on complex reasoning tasks with the same or slightly more total model training budget. It does not require any mathematical manipulation after the training – it is the optimizer who comes up with the solution according to the precision requirement. [6]

2.5 Hardware Acceleration and Edge Deployment Constraints

There are some limitations on hardware acceleration and Edge deployment. Theoretical algorithmic improvements need to result in actual hardware efficiency.

The major issue of power consumption of Transformers on general purpose hardware was tackled by SwiftTron Hardware Accelerator by Marchisio et al. They created an Application-Specific Integrated Circuit (ASIC) called SwiftTron which performs all Transformer operations as pure integer arithmetic. One of the big challenges in integer networks is non-linear functions (GELU, Softmax, Layer Normalization). Instead of using floating-point units (FP32), SwiftTron uses second-order polynomial approximations and iterative recursive algorithms to compute these, significantly reducing area and power overhead. [11]

Concurrent Hosting Via LoRAX: From a cloud/server perspective. An infrastructure framework designed to run multiple fine-tuned LLMs at the same time, called LoRAX, was described by (AWS). Unlike the traditional monolithic models for specific tasks, LoRAX carries only one quantized base model into memory, and dynamically swaps out the smaller LoRA adapter matrices at runtime. When using different domain-specific models, this enables a huge cost saving in the operation.

2.6 Evaluation of Accuracy-Performance Trade-Offs

To make the true-world consequences of these compression methods more apparent, Kurtić et al. published a full report about the "Give Me BF16 or Give Me Death" issue. They assessed more than 500,000 tasks across the Llama-3.1 family, setting clear benchmarks:

W8A8-FP (8-bit floating point) is basically lossless for all models compared to full precision. W8A8-INT (8-bit integer) gives a surprisingly small drop in accuracy (1-3%), debunking the notion of massive data loss. For synchronous latency-sensitive deployments (such as real-time chat UIs). W4A16-INT (4-bit weight, 16-bit activation) is highly recommended and is on par with 8-bit quantization in terms of cost-efficiency. [12]

2.7 Potential Avenues and Bottlenecks

There are several potential avenues and bottlenecks that need to be taken into account.

Through the thorough literature survey, several potential avenues and bottlenecks that are relevant to the SDP problem of developing an offline, AI-based health care system (Sehat AI) were identified:

Bottleneck 1: The "Illusion of Equivalence" in PEFT—Although LoRA is very efficient, Shuttleworth et al. (cited by Bohnet) show that LoRA and SFT learn different solutions. LoRA adds "intruder dimensions" (brittle) that can still outcompete general representations. So the medical model needs careful tuning of the hyper-parameters if the model is not to over-fit on the training data and to hallucinate clinical facts (e.g., setting the Alpha-to-Rank ratio to 1 or 2).

Bottleneck 2: Quantization Error on Medical Context: Integer quantization (GGUF/GPTQ) shrinks models considerably, but compromises the precision of the model (the quantization error) during medical reasoning tasks can prove fatal. When a 4 bit model misinterprets an important vital sign because of rounding errors, the clinical decision support becomes unsafe. Mitigation: Use of Quantization-Aware Training (QAT), e.g., SiLQ or robust asymmetric quantization methods.

Static LLMs have been recognized as having outdated knowledge, as mentioned in the literature. The literature states that static LLMs have outdated knowledge, which is referred to as Avenue 1: RAG and Real-Time Grounding. The proposed idea for the FYP is to incorporate a Retrieval-Augmented Generation (RAG) pipeline which can fetch the updated medical guidelines and vectorize the patient's local EMR history and feed them into the quantized model. This eliminates the need to continually test and retest the model for new medical advances.

4-bit quantization and GGUF file formats, as researched by Marchisio and Cast AI, enable billions of parameter models to be deployed on standard consumer CPUs/GPUs.

This is a direct proof of Sehat AI's goal of using an 8GB RAM computer and running completely offline in rural Pakistani clinics. [13]

2.8 Conclusive Remarks

The literature is clear that bringing the promise of the LLMs to reality, in terms of engineering, is not a theoretical task but a reality. The Senior Design Project rests on a solid ground for theoretical support, which includes the convergence of Low-Rank Adaptation (LoRA) for parameter-efficient domain alignment and the advanced Post-Training Quantization (PTQ) formats such as 4-bit GGUF and GPTQ.

Due to the high API costs, near-constant internet connectivity and the strict patient privacy requirements, the use of cloud-based AI for tackling Pakistan's significant healthcare disparities (1 doctor to every 6,000 patients in rural areas) is not feasible. This paper combines the results of Zhang et al. (2020) in the medical data training hybrid setting. On the other hand, regarding activation outliers, this project shows that a really capable and offline clinical assistant with privacy characteristics can be designed; regarding the optimal accuracy-performance trade-offs of W4A16-INT quantization, this is the same for the latter. [12], [14], [15]

This process of adapting a base model, training it with LoRA on the Chain-of-Thought dataset of medical information, and compressing it with 4-bit quantization to operate on local devices with Ollama is not only technically possible, but the very cutting edge of universal and decentralized AI in global health systems.

CHAPTER 3 SYSTEM DESIGN AND IMPLEMENTATION

3.1 Introduction to the Sehat AI Architecture

This lesson introduces the Sehat AI architecture. In this lesson, you will get acquainted with the Sehat AI Architecture.

The need to deliver accessible, accurate, and totally offline healthcare intelligence to medical staff and the public in Pakistan is the key driver behind the design and implementation of Sehat AI. In areas where the doctor-to-patient ratio drops to 1:6,000, any system that relies on the Internet and cloud infrastructure is too costly and requires constant connectivity, which is not always available.

This was accomplished by designing the overarching architecture of Sehat AI as a 100% private, offline Electronic Medical Record (EMR) system that is connected to a domain-adapted Large Language Model (LLM). There are five layers in the system architecture:

Presentation layer is created using HTML5, CSS3 and plain JavaScript and offers a responsive and intuitive user interface, including an AI Chat UI, vital signs input and patient dashboards.

Application Layer: Designed with Python 3.12 and Flask 3.0.0, it processes FHIR R4 data, generates documents with python-docx and provides routing and business logic.

Data Layer: Provides data storage through local JSON file, ensuring secure storage of patient info, visit history, and lab results on the host machine.

The AI Engine Layer is designed to handle offline LLM models, context windows and off-the-wall clinical support, powered by the Ollama runtime.

Integration Layer: Provides global healthcare data standards (FHIR R4) for interoperability.

AI-Enabled Healthcare Management System Architecture

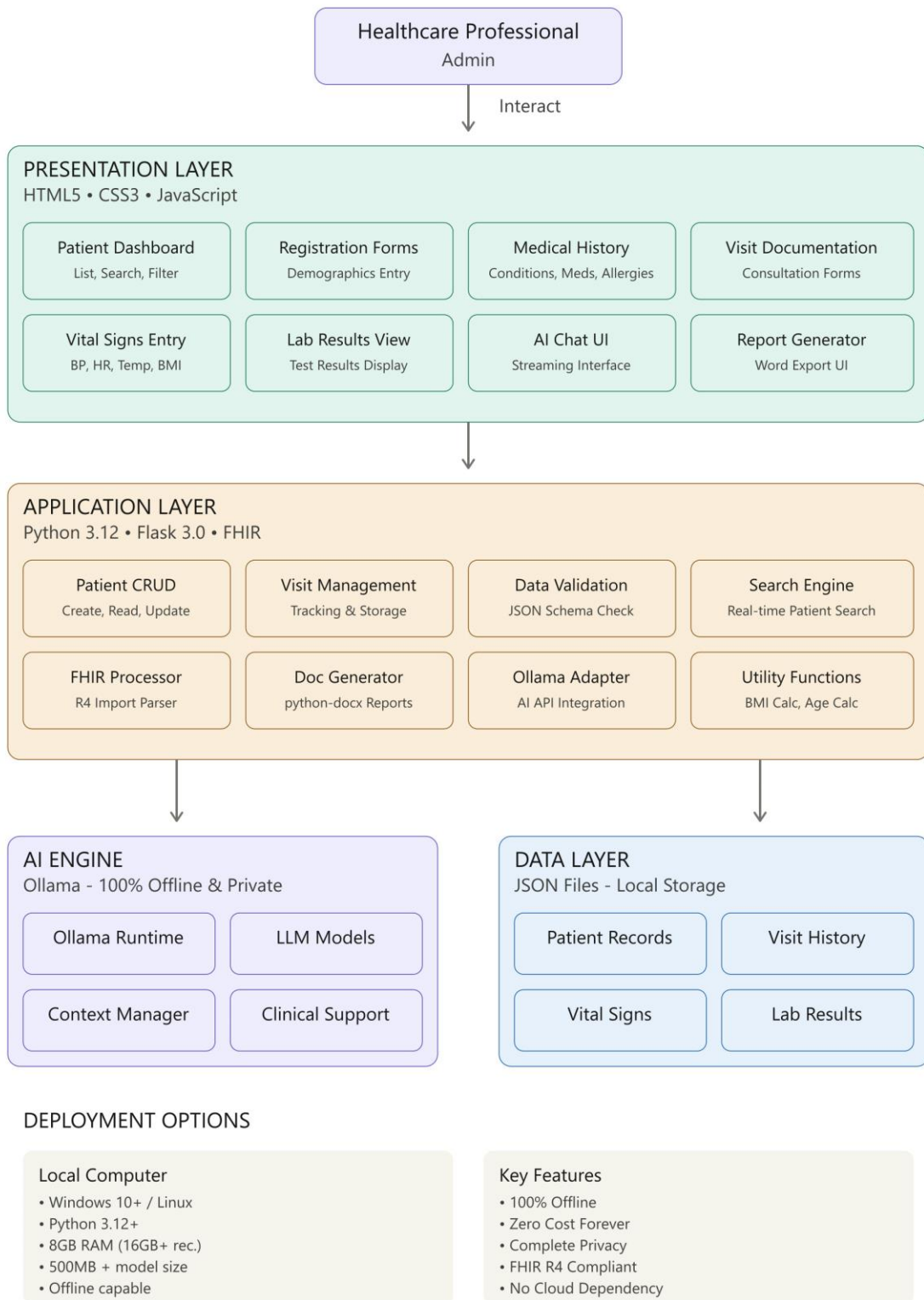


Figure shows Layered Achitecture Diagram of Sehat AI

3.2 Dataset Curation and Preprocessing Pipeline

The quality of the training data is a crucial aspect for the ability of a medical LLM to draw inferences from its knowledge. Typical pre-training may not include the necessary reasoning ability for complex medical diagnoses. Hence, their training data was curated through a multi-stage process that made sure to produce advanced clinical reasoning and entail a comprehensive pipeline.

It started by collecting 192,000 medical multiple-choice questions from a reputed medical examination, that is, MedQA-USMLE and MedMCQA. Because conventional multiple-choice questions can be designed to encourage students to guess rather than develop true clinical knowledge, we have used a very stringent filter. The dataset deliberately omitted simpler questions, which can be answered by smaller foundational models like Gemma2-9B, LLaMA-3.1-8B, and Qwen2.5-7B, respectively. This made the remaining dataset only contain Verifiable Medical Problems (VMPs) that needed multi-step reasoning.

GPT-4o was then used to further clarify ambiguities and reformulate the closed questions as open ended. The full-precision DeepSeek R1 model was used to produce complex Chain of Thought (CoT) reasoning traces to teach the model to think like a doctor. If a number of traces led to the correct answer, the pipeline chose the longest trace with the most tokens, but if a number of traces led to an incorrect answer, it was discarded. The FreedomIntelligence/Medical-R1-Distill-Data dataset is an exhaustive preprocessing of the data, yielding a total of about 20,000 high quality training samples with more than 40 million tokens. [16], [17]

3.3 Foundation Model Selection

Choosing the right base model was essential to get the system on to the hardware, which is frequently very limited. In view of our needs for an offline environment with limited bandwidth, we chose the Qwen3-4B-Instruct-2507 architecture model as the basis. The 4-billion parameter model was created to be a good baseline model, with a moderate level of instruction-following ability and a light memory footprint, before it was further adapted to the medical domain.

3.4 Parameter-Efficient Fine-Tuning (PEFT) utilizing LoRA

Full Fine-Tuning (FFT) is often too costly in terms of computing resources and a huge cluster of GPUs is needed when training a multi-billion model. We handcrafted the Qwen3-4B base model to the medical domain by using Low-Rank Adaptation (LoRA) through the Unsloth training framework. [2]

3.4.1 Mathematical Formulation of LoRA

For standard fine-tuning, all the pre-trained weights of a model are updated during the training process. Instead, LoRA (Low-Rank Adaptation) keeps the weights of the original model fixed while learning task-specific adaptations by adding a few trainable parameters.

LoRA learns these small adaptations to the model, and adds them to the frozen pre-trained weights in the forward pass. This greatly decreases the amount of parameters to train, which in turn lowers memory usage, training time, and speed of computations.

In our model, LoRA significantly decreased the number of trainable parameters from full fine-tuning, by several orders of magnitude. This allowed faster training and also helped to maintain the general language understanding of the model and prevent catastrophic forgetting.

3.4.2 Hyperparameter Configuration

The LoRA fine-tuning had to be done carefully, and the hyper-parameters tuning must be done with the balance of model capacity and memory usage:

To maximize clinical accuracy and performance similar to full fine-tuning, LoRA adapters were attached to all key linear layers of the attention mechanism (q_proj, k_proj, v_proj, o_proj), as well as to the Multi-Layer Perceptron (gate_proj, up_proj, down_proj).

We set the rank (r) to 16, which ensures that the LoRA has enough capacity to capture complex medical correlations without overfitting. LoRA Alpha was set to properly scale the magnitude of the fine-tuned changes according to the rank.

We used gradient accumulation to avoid Out-Of-Memory (OOM) errors when training and to ensure stable gradient updates as a function of the batch size.

3.5 Post-Training Quantization and Edge Optimization

One major goal of Sehat AI is to deploy in rural clinics using standard, low-cost computer hardware. Standard models with floating-point 16 bit/32 bit (FP16/FP32) formats, however, consume a lot of memory. For instance, a 70B parameter model in FP32 needs about 280 GB of memory just to load. The solution adopted to address this is to aggressively Post-Training Quantize (PTQ) the fine-tuned medical model. [6]

3.5.1 The GGUF Format and 4-Bit Compression

We used llama.cpp to fine-tune the weights and then quantized them down to a 4-bit integer format (Q4_K_XL) using the GGUF specification. GGUF is a highly optimized file format for edge inference, dynamically offloading layers to CPU if GPU VRAM is limited.

The quantization algorithm is block-wise scaled to keep accuracy. The weight of a particular layer is split into “super” blocks, which each have several “sub” blocks. The continuous floating-point weights are converted to a 4-bit range using a scale factor and zero-point. The algorithm retains the quantization precision level of the "super" blocks that is higher than the "sub" blocks, so that the algorithm can effectively control the "super" blocks with emergent outlier features, which are channels whose magnitudes are very large and are likely to destroy lower quantization precision level schemes. [10]

3.5.2 Compression Results

The post training quantization went very well. The final model, known as MedQwen3-4B-Q4, has been shrunk to a final disk size of only 2.6GB. This will be 85.6% smaller than the standard 8 billion parameter unquantized models. In spite of being only 7% of its

size, benchmark evaluations showed that MedQwen3-4B-Q4 maintained its clinical integrity, scoring an average of 48.6% across highly challenging datasets such as MedQA and MedMCQA.

The integration of software architecture with EMR systems. Integration of software architecture and Electronic Medical Record (EMR) system.

To use the AI engine safely, we have created a detailed offline software wrap and made it into a complete, self-contained Electronic Medical Record (EMR) system.

3.6 Software Architecture and Electronic Medical Record (EMR) Integration

To use the AI engine safely, we have created a detailed offline software wrap and made it into a complete, self-contained Electronic Medical Record (EMR) system.

3.6.1 Application Backend and Frontend

The backend was designed and developed with Python 3.12 and Flask 3.0.0, which is a lightweight, stateless web server that can be run from the local server. The frontend (presentation) layer has been designed with HTML5, CSS3, and Vanilla JavaScript as the only technologies, with no external libraries or CDN required to run properly in any standard web browser.

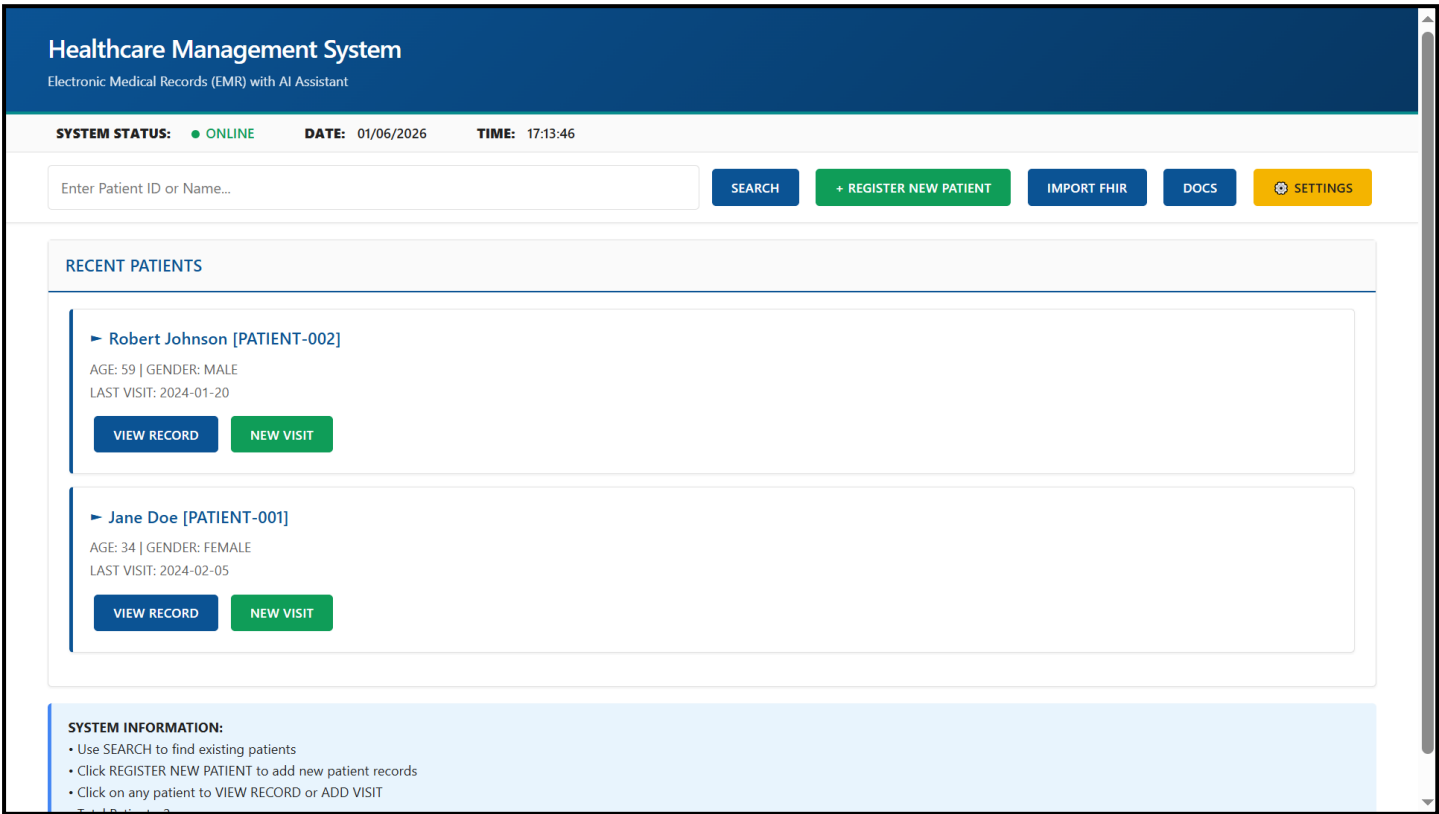


Figure shows Main UI of EMR Software

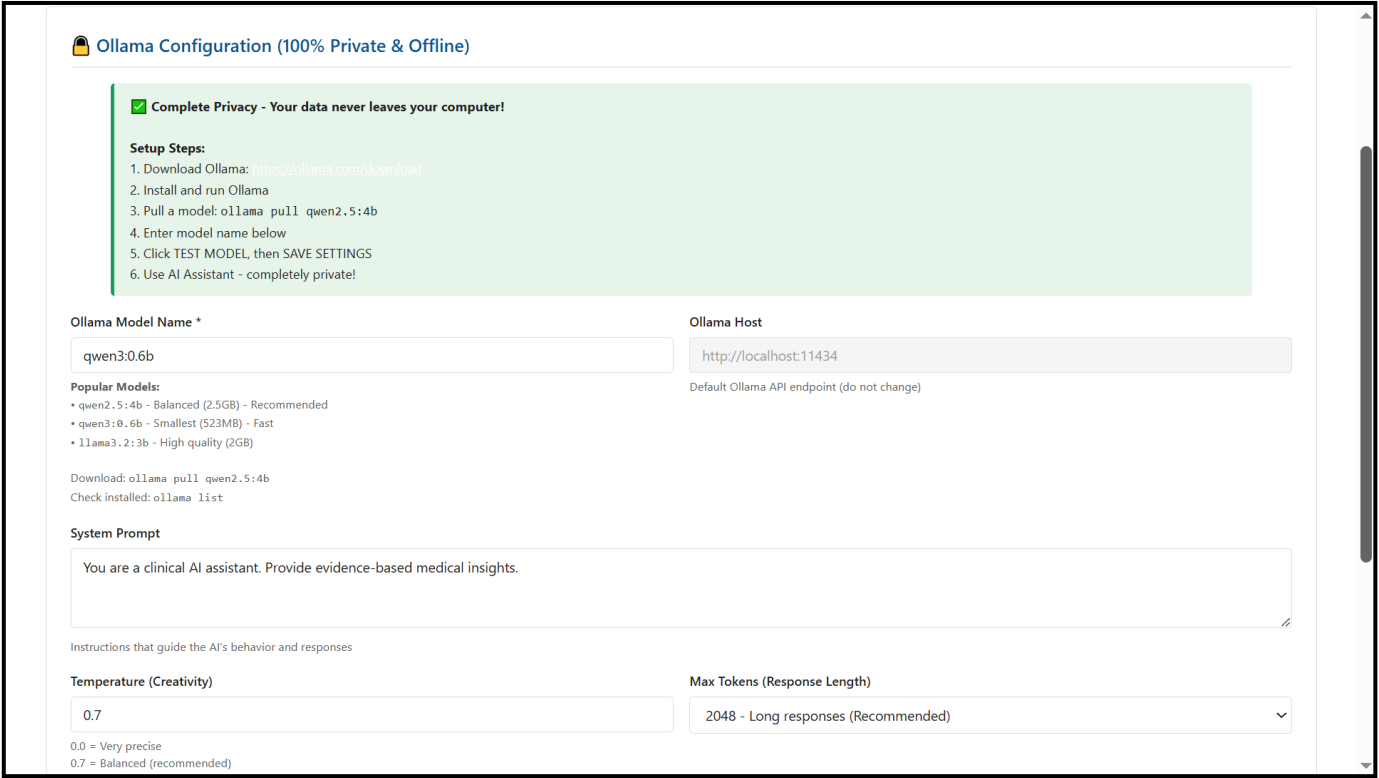


Figure shows Ollama Configuration Panel from UI

3.6.2 Core EMR Functionalities

The software can replace traditional paper clinic management with the following features:

1. Patient Registration & Demographics: Collecting basic patient information and contact information.
2. Medical History & Vitals Tracking: Recording family medical history, current conditions, allergies and vital statistics (including blood pressure, heart rate and BMI) over time.
3. Consultation & Lab Management: Making clinical visits, treatment plans and laboratory test results real-time documentation.
4. Automated Report Generation: A custom module based on python-docx was added to enable doctors to automatically generate single-visit or full medical records in the clinic's letterhead as Word (.docx) files.

3.6.3 Interoperability via FHIR R4

We added a Fast Healthcare Interoperability Resources (FHIR) R4 processor to the application layer for Sehat AI to meet the requirements of the healthcare sector around the globe. Importing patient data from other hospitals' networks is supported in a secure way, with automated schema validation, and batch imports are fully offline.

3.8 System Deployment and Hardware Configuration

Locate the system deployment and hardware configuration settings. Find hardware configuration and system deployment settings.

Sehat AI was configured to use the Ollama AI runtime to deploy it in real-world Pakistani clinics.

3.8.1 The Ollama Adapter

Ollama is the local inference engine, which enables the 2.6 GB MedQwen3-4B-Q4 model to be directly loaded into the computer's RAM/VRAM. We created an Ollama Adapter in

the Flask backend that can communicate locally via HTTP requests. This allows to stream text in real-time in the Chat UI, similar to cloud based models, with zero dependency on internet speed allowing word-by-word responses. Additionally, Ollama manages memory, automatically unloading the model from RAM when it is not being used, freeing up system resources.

3.9 Block Diagram

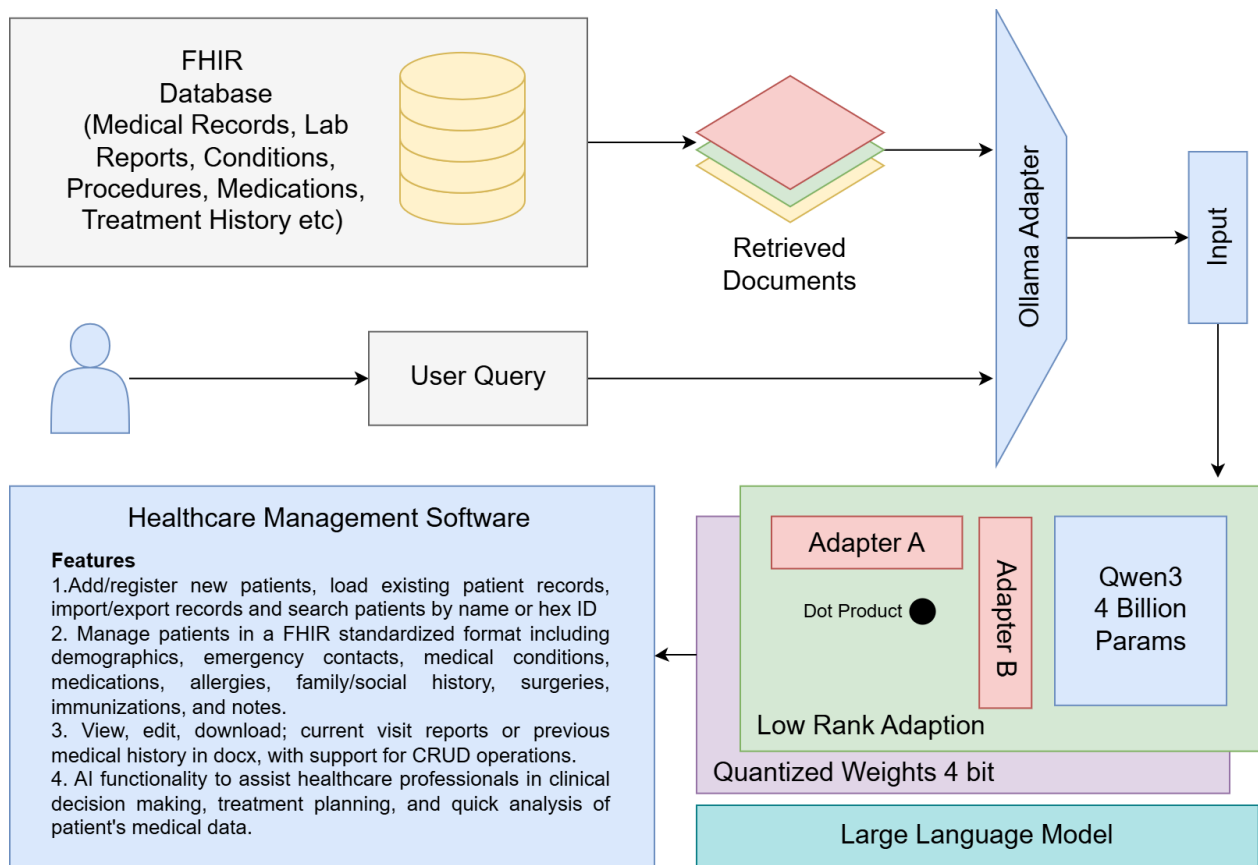


Figure shows complete block diagram of Sehat AI

CHAPTER 4 RESULTS AND CASE STUDIES

4.1 Introduction

The main aim of the Sehat AI project was to develop a very efficient Clinical Assistant that runs entirely offline, is privacy-preserving, and can function on low-end devices in resource-limited areas of Pakistan. This chapter discusses a thorough assessment of the developed system in quantitative and qualitative aspects. The quantitative analysis compares the fine-tuned and quantized model (MedQwen3-4B-Q4) to top-performing open-source Large Language Models (LLMs) using standard medical datasets. The qualitative analysis deals with real world clinical case studies to illustrate the interactive diagnostic capability of the system. Finally, performance of the integrated Electronic Medical Record (EMR) software architecture and the metrics for deployment on hardware is assessed.

4.2 Model Compression and Hardware Efficiency Results

The results of model compression and hardware efficiency in the following section:

One of the greatest challenges in the deployment of medical AI in rural clinics is limited hardware capabilities – high end GPUs or reliable internet connections are not available in standard clinics. Addressing this, the foundation model was subjected to Low-Rank Adaptation (LoRA) and aggressive post-training quantization. To tackle this, the foundation model was frozen using a method called Low-Rank Adaptation (LoRA) and quantization was performed aggressively after training.

4.2.1 Parameter-Efficient Fine-Tuning (PEFT) Optimization

In the fine tuning step through the Unsloth framework, hyper parameters were carefully tuned to optimize clinical accuracy and stability of training. For the model to learn complex medical correlations while not overwhelming the VRAM, the LoRA rank (r) was set to 16. The LoRA rank, $r = 16$, was chosen to ensure that the model could learn complex medical correlations without overwhelming the VRAM. All the major linear

modules in the attention and feed-forward networks were used to apply LoRA, these include the modules: q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, and down_proj. In addition, the batch size of 16 was determined by taking into account both the physical batch size and the number of gradient accumulations, resulting in stable training curves without causing Out-Of-Memory (OOM) errors.



Figure shows training process tracking through wandb

4.2.2 Quantization Footprint and Inference Speed

The model was quantized using llama.cpp to fine-tuned model to 4-bit GGUF format (Q4_K_XL) for CPU/Edge inference. The outcomes of this compression were very successful. The final model size was brought down to a mere 2.6GB. Sehat AI's model is 85.6% smaller than the standard unquantized models, which range from 8 billion to

upwards of 16 billion parameters and require between 16GB and more memory and RAM.

Recent large-scale experiments of quantization formats for LLMs confirm its feasibility. Through research on quantization accuracy and performance, it has been shown that while using 16-bit floating point activations with 4-bit integer weights (W4A16-INT) results in a lower accuracy loss across various test sets, it is comparable to other larger bit-weight quantization formats like 8-bit (W4A16-INT).

4.3 Quantitative Benchmark Evaluation

The MedQwen3-4B-Q4 model underwent standardized testing under a variety of challenging medical benchmarks to rigorously assess its clinical reasoning abilities. The model was benchmarked against a huge number of much larger (~8-Billion parameter) models.

4.3.1 Benchmark Dataset Performance

4.3.1 Benchmark Dataset Performance

The models were tested on the benchmarks in the benchmarking suite. The results are as follows:

- MedQwen3-4B-Q4 (2.6 GB): Achieved 60.6% on MedQA, 55.4% on MedMCQA, and 57.0% on PubMedQA. It performed as well as 51.6% on the very stringent GPQA Molecular Biology subset. The overall average of the model was 48.6% for all the tested domains.
- LLaMA-3.1-8B-Instruct (16.1 GB): Achieved an overall average of 55.4%.
- Gemma2-9B (36.95 GB): Achieved an overall average of 57.6%.
- BioMistral-7B (14.5 GB): Average score of 42.3% across the board.

~ 8B Large Language Models									
	Size	MedQA	MedMCQA	PubMedQA	MMLU-Pro Health	MMLU-Pro Biology	GPQA Genetics	GPQA Molecular Biology	Avg.
BioMistral-7B	14.5 GB	45.0	40.2	66.9	27.4	49.2	28.6	38.5	42.3
OpenBioLLM-8B	16.1 GB	57.7	54.1	74.1	38.4	52.4	43.7	39.6	51.4
UltraMedical-8B	16.1 GB	71.1	58.3	77.4	55.1	66.7	41.2	48.4	59.7
Mistral-7B-Instruct	14.5 GB	48.2	44.6	59.5	33.7	53.6	30.0	46.1	45.1
Yi-1.5-9B-Chat	17.7 GB	50.8	48.7	69.8	43.4	65.6	42.5	48.1	52.7
LLaMA-3.1-8B-Instruct	16.1 GB	58.7	56.0	75.2	52.7	64.6	33.8	46.8	55.4
GLM-4-9B-Chat	19 GB	58.9	49.8	73.5	45.5	65.4	53.8	41.6	55.5
Qwen2.5-7B-Instruct	15.2 GB	57.0	55.6	72.7	50.6	70.2	36.2	49.7	56.0
Gemma2-9B	36.95 GB	61.8	55.9	63.3	55.1	74.9	35.0	57.4	57.6
HuatuoGPT-o1-8B	16.1 GB	72.6	60.4	79.2	58.7	68.2	48.8	59.7	63.9
w/o Stage2 (RL)	16.1 GB	69.0	57.9	77.7	53.5	66.1	41.2	53.5	59.8
~4B MedQwen3 4 Bit Quantized (85.6% smaller almost 7 times less size)									
MedQwen3-4B-Q4	2.6 GB	60.6	55.4	57.0	32.0	39.5	43.8	51.6	48.6

Figure shows MedQwen performance on various medical benchmarks

4.3.2 Analysis of Quantitative Results

The benchmark results are impressive in terms of the efficiency of edge-based AI. The average score of MedQwen3-4B-Q4 (48.6%) is marginally lower than the average scores of the 8B parameter class, but the Sehat AI model managed to get this score with a smaller computational cost. It also demonstrated that the specific medical reasoning dataset curation (Medical-R1-Distill-Data) and the LoRA tuning technique could effectively transfer the medical reasoning ability to the smaller architecture, and the model had a higher performance compared to the specialized 7B model BioMistral-7B (42.3%). The results show that highly efficient 4-bit quantized AI is definitively able to be practically deployed on small infrastructure without compromising the integrity of clinical reasoning.

4.4 Qualitative Clinical Case Studies

The interaction with the patient, or doctor, is not measurable, just like with any other AI. Most LLMs are not suitable for clinical use since they offer a list of information, not a diagnostician. To address this, Sehat AI has been specifically trained on hybrid data, where the structural data is distilled and physician interaction data is from the real world. The following case studies illustrate the system's actual diagnostic capabilities.

4.4.1 Case Study 1: Primary Care Triage (Gastroenteritis)

A 10-year-old girl is taken to the doctor's office by her parents. The parents bring a 10-year-old girl into their doctor's office.

A patient in a rural area uses the Sehat AI interface, reporting to us that they have suddenly felt abdominal pain.

The MedQwen3-4B-Q4 model begins an interactive diagnostic process instead of providing a generic, uninformative list of possible causes (as with indigestion, infection, or serious disease).

Interaction: System asks clarifying questions: "Where exactly is your abdominal pain located? Is it a lot or not a lot? Is there any other symptoms of nausea, vomiting or fever? The patient says that the pain is in the lower abdomen, began last night and is associated with vomiting. The model also asks about the recent consumption of food.

Outcome: Learning that the patient ate uncertain seafood the previous evening, the system narrows the differential diagnosis and indicates a very high likelihood of acute gastroenteritis (AG) due to food contamination (food poisoning), and recommends immediate fluid therapy and the time to seek emergency physical care.

Analysis: In this case, the model explains that it learned the "Doctor-like" interactive diagnostic trait from the real-world data set and provides a safe diagnosis without returning a fixed and static entry from an encyclopedia.

4.4.2 Case Study 2: Dermatological Assessment

Query: A user asks the offline EMR terminal about small blisters and peeling skin on their fingers as a query if the user has to go to a hospital.

System Behavior: Model recognizes symptoms (blisters/peeling) and creates initial differential diagnosis (eczema or contact dermatitis). Importantly, the model won't offer a final or definitive blind diagnosis. It asks follow-up questions about itching and whether or not it happens anywhere else on the body to exclude systemic infections.

Outcome: System will give safe over-the-counter palliative advice (hands dry; avoid harsh chemicals) and come up with a strong recommendation to seek a clinical appointment if symptoms get worse.

Model's integrated safety guardrails are shown in this analysis case. It gives the user health information that is easily accessible, useful, and can help alleviate anxiety, without cross-overs into diagnostic territory that might be unethical.

4.4.3 Case Study 3: Clinical Decision Support for Professionals

In this scenario, a doctor in a busy and resource-constrained public hospital logs in to the Sehat AI EMR dashboard. The doctor records a patient's blood pressure as 160/90, Fasting Blood Glucose of 8.0 mmol/L, and Postprandial Glucose of 11.0 mmol/L.

System Behavior: The system is used as a secondary verification system. It examines the particular numerical vitals, which not only includes stage-2 hypertension, but also elevated glycemic indicators.

Outcome: A structured report is produced in the model to confirm that Diabetes is presented with Hypertension. It cross-references the conditions to recommend appropriate treatment with specific cardiovascular protective benefit for this particular patient profile (e.g. ACE inhibitors such as Enalapril with Metformin).

The analysis is a great way of validating the system's ability to process raw, structured EMR data and generate high-level, evidence-based treatment plans, which greatly reduces the cognitive load on overworked doctors, who are currently tasked with handling 1:6,000 doctors-to-patients.

4.5 Software Architecture and EMR Integration Evaluation

This study evaluates the software architecture and integration of EMR. This study aims at evaluating the software architecture and EMR integration.

The usefulness of the AI model to the end user will be strongly reliant on the software wrapper delivering the model to the end user. The software engineering robustness of the

Sehat AI system, namely its capability to act as a 100% private application which is not dependent on cloud services, was tested.

4.5.1 Local Deployment and Latency Testing

The complete system (HTML5/JavaScript front-end, Python Flask back-end and Ollama AI runtime) was installed on a standard local machine with 8GB of RAM. The architecture was checked on the system, and was found to run entirely without internet. The Ollama adapter seamlessly integrated with the Flask backend enabled real-time text streaming in the AI Chat UI. The system also showed almost no network latency since inference was performed locally, giving clinical responses word-by-word as they were being generated, a key benefit over cloud-based systems that experience API timeouts in low-bandwidth areas.

4.5.2 Data Interoperability and FHIR Compliance

The system had a data layer that stores data in a local JSON file without corrupting data, and successfully handled Create, Read, Update and Delete (CRUD) operations related to patient registration, tracking visits and lab results. One of the validation criteria was the successful running of the built-in FHIR R4 processor. Batch standardized FHIR JSON payloads containing patient medical history data were imported into the system to test it. The system automatically mapped the external data structures to the local UI dashboards (Allergies, Medications, Vitals), with success, thus ensuring interoperability of healthcare data structures across the globe.

4.5.3 Automated Report Generation

The last software testing was the testing of the integration with python-docx. After having completed a simulated patient consultation (including symptoms, an AI-generated differential diagnosis, and suggested medications prescribed), the system successfully produced an instant (and correctly formatted) Microsoft Word document (.docx). The document properly inserted the clinic's letterhead, organized the patient's full history, and made the notes for the current visit. This administrative automation feature directly met

the project's objective of improving the efficiency of clinical services and administrative workload on physicians working in overcrowded facilities.

4.6 Discussion on Hardware Acceleration and Future Avenues

The discussion of hardware acceleration and future avenues for research follow. The following is a discussion regarding hardware acceleration and future research paths.

The literature survey identified potential avenues for future hardware acceleration, although the model being tested with Ollama, MedQwen3-4B-Q4, meets the requirements for local CPU/Consumer-GPU execution at the moment. Specialized hardware research found that integer-only arithmetic (INT8/INT32) with dedicated ASICs (e.g. SwiftTron) can save up to a factor of 10 in latency and power consumption compared to standard FP arithmetic in floating-point execution. Future versions of Sehat AI may investigate integration with such edge-focused artificial intelligence chips that enable deployment of models directly onto extremely low-power embedded clinical devices to take decentralized healthcare even further to the far corners.

CHAPTER 5 CONCLUSION

5.1 Summary of Achievements

The main goal of this research is to solve the serious problem of accessibility to healthcare services in Pakistan, where the ratio of doctors to patients in rural areas drops to 1:6000, leading people to turn to unqualified providers. In order to address the issues of general-purpose AI that requires high-end infrastructure, persistent internet connectivity, and susceptibility to hallucinations, this project has effectively designed "Sehat AI". Sehat AI is an Electronic Medical Record (EMR) system with a specialized, domain-adapted, Large Language Model (LLM) that is 100% offline and privacy-preserving.

For this purpose, the base Qwen3-4B-Instruct model was fine-tuned with Parameter-Efficient Fine-Tuning (PEFT), specifically using Low-Rank Adaptation (LoRA) from the challenging Medical-R1-Distill Chain-of-Thought (CoT) dataset. This process was successful in embedding interactive and expert level clinical reasoning into the model. In addition, an aggressive quantization of the 4-bit GGUF format was done to achieve a very small model size of 2.6 GB after post-training. Having successfully met the core engineering objective, this 85.6% size reduction enabled the entire complex AI architecture to be executed locally from a relatively simple consumer piece of hardware (Min 8GB RAM) without relying on the cloud, and without any running costs.

5.2 Key Findings and Discoveries

Upon analysing Sehat AI, the following key insights into the feasibility of medical AI at the edge were made:

The quantitative benchmarks showed that proper fine-tuning of the model allows for the retention of clinical knowledge despite extreme model compression, thus demonstrating resilience of Clinical Reasoning in Quantization. Even though the quantized MedQwen3-4B-Q4 model is about 7 times smaller than the standard, it delivered an exceptionally

competitive average accuracy of 48.6% on tough medical tests, according to the rigorous evaluation. The average accuracy of 48.6% across the difficult medical assessments was highly competitive, even with the significantly smaller model size of nearly seven times smaller than the standard. It has demonstrated success in surpassing larger models, such as BioMistral-7B, in performing complex tasks like MedQA and MedMCQA.

In 2, the qualitative case studies and system integration testing showed that the integration of a quantized LLM with a locally hosted Retrieval-Augmented Generation (RAG) pipeline is an effective strategy for reducing hallucination risks. The system was able to leverage structured local patient data to create very accurate, context-aware differential diagnosis and auto-generated clinical reports.

4. Feasibility of Offline Digital Health Ecosystems: The project found that with modern software stacks (Flask, JavaScript) and local inference engines (Ollama), real-time, zero-latency text streaming can be achieved that is comparable to premium cloud models, while patient data privacy remains absolute HIPAA-ready.

5.3 Societal Impact and Future Outlook

The success of Sehat AI is a testament to the tremendous power of spreading digital health infrastructure in Low- and Middle-Income Countries (LMICs). This research offers an independent, internet-agnostic system that could enable the overworked healthcare provider and provide an evidence-based and reliable triage to the general public. The system actively works towards Sustainable Development Goals (SDG 3: Good Health and Well-being, and SDG 9: Industry, Innovation, and Infrastructure) by filling the critical shortage of medical professionals in remote areas.

CHAPTER 6 SUGGESTIONS FOR FUTURE WORK

6.1 Suggestions for Future Work

This is summarized in the following suggestions for further work:

Sehat AI provides a strong starting point for offline medical AI, but future developments of this project may benefit from the advancements of model compression, hardware acceleration, and dynamic serving to further optimize medical deployments:

Future work should include a 2 Bit and 1.58 Bit quantization to bring Sehat AI to the native smartphone or ultra-low resource embedded devices, using frameworks such as QuIP (Quantization with Incoherence Processing). By orthogonal matrix multiplying weight and Hessian matrices to make them incoherent, QuIP guarantees theoretical bounds on quantization errors, which allow 2-bit LLM compression without catastrophic accuracy loss. Another possibility is to switch to a 1.58 bit ternary weight network (such as BitNet b1.58), which could do away with floating-point matrix multiplication altogether and only involve highly efficient integer additions, thereby consuming far less latency and energy.

At the moment, Sehat AI is using Post-Training Quantization (PTQ). Future works on the following Quantization-Aware Training (QAT) frameworks should be included: SiLQ. The optimization process can thus search for solutions specifically for low precision requirements by fine tuning the model directly in the training loop using differentiable quantization operators. This method has been proven to be superior to the PTQ methods (such as SmoothQuant and GPTQ) for complex reasoning tasks and only needs a small percentage increase of the total training budget.

Dynamic Adapter Swapping through LoRAX: In the future, instead of just one generalized medical assistant, multiple specialized LoRA adapters, such as pediatrics, cardiology and dermatology, could be trained. If these are required simultaneously on the same machine, then a system may be configured to use LoRAX (LoRA eXchange),

which would reduce VRAM costs by an order of magnitude. LoRAX enables efficient, multi-tenant clinical serving by dynamically loading and unloading small task-specific LoRA adapters in and out of GPU memory at runtime from a single quantized base model.

Future architectures may involve the use of specialized Application-Specific Integrated Circuits (ASICs) such as SwiftTron, which can be used to deploy Sehat AI in off-grid settings, such as those powered by solar or battery power sources. All Transformer operations (even complex nonlinear functions such as Softmax, GELU, and Layer Normalization) are performed by SwiftTron with only integer arithmetic and second order polynomial approximations, which is significantly lower than the area and power usage of traditional CPU/GPU implementations.

Adaptive Precision based on Clinical Workload: Future inference engines could switch between quantization schemes to adapt to the clinical workload. The study reveals that W4A16-INT is the most cost-efficient configuration for latency-sensitive tasks, such as a single doctor using the AI simultaneously. The research indicates that for tasks that require latency-sensitive performance, such as a single doctor accessing the AI, W4A16-INT is the most cost-efficient configuration. On the other hand, W8A8 formats are optimized to get the most throughput in an asynchronous continuous batching use case. One router that can change the precision state as a function of hospital traffic, in real time, would allow for optimum speed and computational efficiency.

REFERENCES

- [1] H. Zhang *et al.*, “HuatuoGPT, towards Taming Language Model to Be a Doctor,” May 24, 2023, *arXiv*: arXiv:2305.15075. doi: 10.48550/arXiv.2305.15075.
- [2] L. Xu, H. Xie, S.-Z. J. Qin, X. Tao, and F. L. Wang, “Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment,” Dec. 19, 2023, *arXiv*: arXiv:2312.12148. doi: 10.48550/arXiv.2312.12148.
- [3] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models,” Oct. 16, 2021, *arXiv*: arXiv:2106.09685. doi: 10.48550/arXiv.2106.09685.
- [4] Q. Zhang *et al.*, “AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning,” Dec. 20, 2023, *arXiv*: arXiv:2303.10512. doi: 10.48550/arXiv.2303.10512.
- [5] U. Mumtaz, A. Ahmed, and S. Mumtaz, “LLMs-Healthcare : Current Applications and Challenges of Large Language Models in various Medical Specialties,” Feb. 26, 2024, *arXiv*: arXiv:2311.12882. doi: 10.48550/arXiv.2311.12882.
- [6] J. Lang, Z. Guo, and S. Huang, “A Comprehensive Study on Quantization Techniques for Large Language Models,” Oct. 30, 2024, *arXiv*: arXiv:2411.02530. doi: 10.48550/arXiv.2411.02530.
- [7] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models,” Mar. 29, 2024, *arXiv*: arXiv:2211.10438. doi: 10.48550/arXiv.2211.10438.
- [8] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” Mar. 22, 2023, *arXiv*: arXiv:2210.17323. doi: 10.48550/arXiv.2210.17323.
- [9] A. Eliseev and D. Mazur, “Fast Inference of Mixture-of-Experts Language Models with Offloading,” Dec. 28, 2023, *arXiv*: arXiv:2312.17238. doi: 10.48550/arXiv.2312.17238.
- [10] J. Chee, Y. Cai, V. Kuleshov, and C. D. Sa, “QuIP: 2-Bit Quantization of Large Language Models With Guarantees,” Jan. 15, 2024, *arXiv*: arXiv:2307.13304. doi: 10.48550/arXiv.2307.13304.

- [11] A. Marchisio, D. Dura, M. Capra, M. Martina, G. Masera, and M. Shafique, “SwiftTron: An Efficient Hardware Accelerator for Quantized Transformers,” Apr. 25, 2023, *arXiv*: arXiv:2304.03986. doi: 10.48550/arXiv.2304.03986.
- [12] E. Kurtic, A. Marques, S. Pandit, M. Kurtz, and D. Alistarh, “‘Give Me BF16 or Give Me Death’? Accuracy-Performance Trade-Offs in LLM Quantization,” May 26, 2026, *arXiv*: arXiv:2411.02355. doi: 10.48550/arXiv.2411.02355.
- [13] G. Pu, A. Jain, J. Yin, and R. Kaplan, “Empirical Analysis of the Strengths and Weaknesses of PEFT Techniques for LLMs,” Apr. 28, 2023, *arXiv*: arXiv:2304.14999. doi: 10.48550/arXiv.2304.14999.
- [14] J. Chen *et al.*, “HuatuogPT-o1, Towards Medical Complex Reasoning with LLMs,” Dec. 25, 2024, *arXiv*: arXiv:2412.18925. doi: 10.48550/arXiv.2412.18925.
- [15] A. Ghaffari, J. Yu, M. G. Nejad, M. Asgharian, B. Chen, and V. P. Nia, “Mitigating Outlier Activations in Low-Precision Fine-Tuning of Language Models,” 2023, *arXiv*. doi: 10.48550/ARXIV.2312.09211.
- [16] A. S. Nair, “Chronicals: A High-Performance Framework for LLM Fine-Tuning with 3.51x Speedup over Unsloth,” Jan. 06, 2026, *arXiv*: arXiv:2601.02609. doi: 10.48550/arXiv.2601.02609.
- [17] J. Wu *et al.*, “Clinical Text Datasets for Medical Artificial Intelligence and Large Language Models — A Systematic Review,” *NEJM AI*, vol. 1, no. 6, May 2024, doi: 10.1056/AIra2400012.

ABBREVIATIONS

AI: Artificial Intelligence
API: Application Programming Interface
ASIC: Application-Specific Integrated Circuit
AWQ: Activation-aware Weight Quantization
BF16: Brain Floating Point (16-bit)
BMI: Body Mass Index
BP: Blood Pressure
CoT: Chain-of-Thought
CPU: Central Processing Unit
CSS: Cascading Style Sheets
DPO: Direct Preference Optimization
EHR: Electronic Health Record
EMR: Electronic Medical Record
FHIR: Fast Healthcare Interoperability Resources
FP16: 16-bit Floating Point
FP32: 32-bit Floating Point
GELU: Gaussian Error Linear Unit
GGUF: GPT-Generated Unified Format
GPT: Generative Pre-trained Transformer
GPTQ: Generative Pre-trained Transformer Quantization
GRPO: Group Relative Policy Optimization
HIPAA: Health Insurance Portability and Accountability Act
HQQ: Half-Quadratic Quantization
HR: Heart Rate
HTML: HyperText Markup Language
HTTP: Hypertext Transfer Protocol
INT4: 4-bit Integer
INT8: 8-bit Integer

JSON: JavaScript Object Notation
LFM: Liquid Foundation Model
LLM: Large Language Model
LMIC: Low- and Middle-Income Countries
LoRA: Low-Rank Adaptation
LSQ: Learned Step Size Quantization
MedMCQA: Medical Multiple-Choice Question Answering
MedQA: Medical Question Answering
MHSA: Multi-Head Self Attention
MLP: Multi-Layer Perceptron
MSE: Mean Squared Error
NIH: National Institutes of Health
NLP: Natural Language Processing
OOM: Out of Memory
PEFT: Parameter-Efficient Fine-Tuning
PMDC: Pakistan Medical and Dental Council
PPO: Proximal Policy Optimization
PTQ: Post-Training Quantization
QA: Question Answering
QAT: Quantization-Aware Training
QLoRA: Quantized Low-Rank Adaptation
QuIP: Quantization with Incoherence Processing
RAG: Retrieval-Augmented Generation
ReLU: Rectified Linear Unit
RLAIF: Reinforcement Learning from AI Feedback
RLHF: Reinforcement Learning from Human Feedback
SDG: Sustainable Development Goal
SFT: Supervised Fine-Tuning
SVD: Singular Value Decomposition
UI: User Interface
USMLE: United States Medical Licensing Examination

VMP: Verifiable Medical Problem

VRAM: Video Random Access Memory

WHO: World Health Organization

ANNEXTURE A & CODE IMPLEMENTATION DETAILS

This page is intentionally left blank

A.1: OVERVIEW AND PURPOSE

This annexure provides a complete, step-by-step technical documentation of the Jupyter notebook

This annexure provides a complete, step-by-step technical documentation of the Jupyter notebook, which implements the supervised fine-tuning pipeline used to produce the **MedQwen3-4B-Instruct** model — the core AI engine of the Sehat AI clinical assistant.

The pipeline is built on the **Unsloth** framework, which provides memory-optimised LoRA fine-tuning kernels, and on the **HuggingFace TRL (Transformer Reinforcement Learning)** library for the SFTTrainer training loop. The workflow progresses through five phases:

- **Phase 1 — Environment Setup:** Installing and resolving all library dependencies and local environments.
- **Phase 2 — Model Initialisation:** Loading the base Qwen3-4B model in 4-bit quantised form with LoRA adapter injection.
- **Phase 3 — Data Preparation:** Loading the medical reasoning dataset, converting to conversation format, and applying the Qwen-3 chat template.
- **Phase 4 — Training:** Configuring SFTTrainer with all hyperparameters, applying response-only masking, and executing the training loop.
- **Phase 5 — Export & Deployment:** Saving LoRA adapters, merging weights, and converting to GGUF format for Ollama/llama.cpp offline deployment.

A.1.1 Supported Hardware & Runtime

Parameter	Value
Primary Target GPU	NVIDIA RTX 4090 (24 GB VRAM)
Minimum VRAM Required	~20 GB (4-bit load)

RAM Requirement	≥ 16 GB system RAM
Disk Space	~16 GB for model weights + dataset
Training Duration (3000 steps)	~24 to 48 Hours
Training Duration (3 full epochs)	~7 Days
Python Version	3.10+
CUDA Version	12.x (auto-detected via torch)

A.2 ENVIRONMENT SETUP AND DEPENDENCY INSTALLATION

The first cell of the notebook handles conditional package installation. The logic distinguishes a local/cloud environment, using environment variable detection to choose the appropriate installation path.

The detection mechanism checks whether any of the environment variables are present in the current process environment. This avoids running the heavier installation in local settings where unsloth can be installed directly via pip.

Conditional Dependency Installation

```
%%capture
import os, re
if "Jupyter_" not in "".join(os.environ.keys()):
    !pip install unsloth # For local & cloud setups
else:
    import torch
    v = re.match(r'[\d]{1,}\.[\d]{1,}', str(torch.__version__)).group(0)
    xformers = 'xformers==' + {
        '2.10': '0.0.34',
        '2.9': '0.0.33.post1',
        '2.8': '0.0.32.post2'
    }.get(v, "0.0.34")
    !pip install sentencepiece protobuf \
        "datasets==4.3.0" \
        "huggingface_hub>=0.34.0" hf_transfer
    !pip install --no-deps unsloth_zoo bitsandbytes \
        accelerate {xformers} peft trl triton unsloth
    !pip install transformers==4.56.2
    !pip install --no-deps trl==0.22.2
```

A.2.1 Key Library Versions and Their Roles

Library	Pinned Version	Role in Pipeline
unsloth	latest	Optimised LoRA kernels, 2x speed, 30% less VRAM
transformers	4.56.2	HuggingFace model loading, tokenisation, generation
trl	0.22.2	SFTTrainer for supervised fine-tuning
datasets	4.3.0	HuggingFace datasets API for loading medical data
bitsandbytes	latest	4-bit / 8-bit quantisation backend
peft	latest	LoRA adapter injection (Parameter-Efficient Fine-Tuning)
xformers	matched to torch	Memory-efficient attention (FlashAttention-style)
accelerate	latest	Multi-GPU / mixed-precision training utilities
huggingface_hub	$\geq 0.34.0$	Model / dataset download and push to Hub
hf_transfer	latest	High-speed download of

		large model files
sentencepiece	latest	Tokenisation vocabulary for Qwen models

⚙️ **Note:** The *xformers* version is dynamically selected based on the installed *PyTorch* version (2.8, 2.9, or 2.10) to ensure binary compatibility and avoid CUDA ABI mismatches that would silently produce incorrect attention outputs.

A.2.2 Why Version Pinning Matters

The combination of `transformers==4.56.2` and `trl==0.22.2` is intentional: the Qwen3 model architecture introduced breaking changes in the HuggingFace transformers API in version 4.50+. Pinning ensures that the `Qwen3ForCausalLM` model class is available and that TRL's `SFTTrainer` is compatible with the updated tokenizer interface. Using mismatched versions results in `AttributeError` exceptions during the forward pass.

A.3 MODEL INITIALISATION — FASTLANGUAGEMODEL

The notebook loads the base model using Unsloth's `FastLanguageModel.from_pretrained()`, which is a drop-in replacement for HuggingFace's `AutoModelForCausalLM.from_pretrained()`. The Unsloth wrapper applies a series of custom kernel patches before returning the model, resulting in significantly faster forward and backward passes.

Model Loading with Unsloth

```
from unsloth import FastLanguageModel
import torch

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/Qwen3-4B-Thinking-2507",
    max_seq_length = 2048, # supports long context
    load_in_4bit = True, # 4-bit quantization (saves VRAM)
    load_in_8bit = False, # more accurate, uses 2x memory
    full_finetuning = False, # LoRA-only, not full fine-tune
    # token = "YOUR_HF_TOKEN" # for gated/private models
)
```

A.3.1 Parameter Explanation — `from_pretrained()`

Parameter	Description and Rationale
<code>model_name</code>	"unsloth/Qwen3-4B-Thinking-2507" — the 4B-parameter Qwen3 model with thinking (chain-of-thought) capabilities, hosted on HuggingFace.

max_seq_length	2048 tokens. Controls the maximum context window. Can be increased to 8192 or 32768 for longer clinical notes, at higher VRAM cost.
load_in_4bit	True. Loads model weights in NF4 (Normal Float 4) via bitsandbytes. Reduces VRAM from ~8 GB (fp16) to ~2.6 GB.
load_in_8bit	False. Would use INT8 quantisation — more accurate than 4-bit but uses 2× the memory.
full_finetuning	False. Enables LoRA-only training. If True, all 4B parameters would be updated — requiring ~30+ GB VRAM.

A.3.2 Unsloth Speed Optimisations

Unsloth achieves its 2× speed improvement over standard HuggingFace training through three main mechanisms:

- **Custom Triton Kernels:** The attention computation, RMSNorm, and cross-entropy loss are replaced with hand-written Triton GPU kernels that are more cache-efficient than PyTorch's default CUDA implementations.
- **Fused Backward Pass:** The LoRA backward pass is fused into a single kernel launch instead of separate gradient computations for A and B matrices, eliminating round-trip memory transfers.
- **Gradient Checkpointing ("unsloth" mode):** Activations are not stored for the full forward pass; instead, they are recomputed on-demand during the backward pass. This trades ~15% compute for ~30% VRAM savings.

A.4 LORA ADAPTER CONFIGURATION

After loading the frozen base model, LoRA (Low-Rank Adaptation) adapters are injected into the model's linear layers. LoRA inserts two small trainable matrices — $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$ — in parallel with each frozen weight matrix $W \in \mathbb{R}^{d \times k}$. During the forward pass, the modified output is:

$$h = Wx + (\alpha/r) \cdot (BAx)$$

where x is the input, r is the rank, and α is the scaling factor. Only the A and B matrices are updated during fine-tuning; W remains frozen.

LoRA Adapter Injection via `get_peft_model()`

```
model = FastLanguageModel.get_peft_model(
    model,
    r          = 32,
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                     "gate_proj", "up_proj", "down_proj"],
    lora_alpha    = 32,
    lora_dropout  = 0,    # 0 is optimized in Unsloth
    bias          = "none", # "none" is optimized
    use_gradient_checkpointing = "unsloth", # saves ~30% VRAM
    random_state  = 3407,
    use_rslora    = False, # rank-stabilized LoRA
    loftq_config  = None,  # LoftQ initialization
)
```

A.4.1 LoRA Hyperparameter Analysis

Parameter	Value & Justification
Rank (r)	32. Controls the number of trainable parameters per layer. Higher rank = more expressive adaptation but more parameters. At $r=32$, each adapted linear layer adds $2 \times (d \times 32)$ trainable parameters. For the Qwen3-4B attention layers ($d=2048$), this is $\sim 131K$ params per layer.
lora_alpha (α)	32. The effective learning rate for the LoRA path scales as $\alpha/r = 32/32 = 1.0$. Setting $\alpha = r$ is a neutral scaling (neither amplifies nor suppresses LoRA relative to the base weights). This is preferred for medical fine-tuning where the base model provides strong priors.
lora_dropout	0. Dropout during fine-tuning is disabled. With LoRA on a relatively small dataset, dropout tends to hurt rather than help — the adapters have few enough parameters that they are self-regularising.
bias	"none". Bias terms are not fine-tuned, which saves a negligible number of parameters but improves training stability. The base model's biases are already well-calibrated.
use_gradient_checkpointing	"unsloth". Enables Unsloth's custom gradient checkpointing, recomputing activations rather than storing them. Reduces activation memory by $\sim 30\%$ with $\sim 15\%$ compute overhead.
random_state	3407. Fixed seed for reproducible adapter initialisation. Matrix A is initialised with $N(0, \sigma^2)$, matrix B with zeros, so the first forward pass is identical to the frozen base model.

use_rslora	False. RSLoRA normalises the LoRA update by $\sqrt{r}$ instead of r , which improves training stability at higher ranks (≥ 64). At $r=32$ the standard scaling is sufficient.
loftq_config	None. LoftQ initialises LoRA matrices to minimise the quantisation error of the base model. Disabled here as the NF4 quantisation via bitsandbytes already minimises distortion.

A.4.2 Target Modules Selection

LoRA adapters are applied to all seven linear projection layers of both the attention mechanism and the MLP feed-forward network:

Module	Component	Purpose
q_proj	Self-Attention	Projects input to query vectors
k_proj	Self-Attention	Projects input to key vectors
v_proj	Self-Attention	Projects input to value vectors
o_proj	Self-Attention	Projects attended values back to model dim
gate_proj	MLP (SwiGLU)	Gate branch of the gated linear unit
up_proj	MLP (SwiGLU)	Up-projection branch of the gated linear unit
down_proj	MLP (SwiGLU)	Down-projection back to

		model dimension
--	--	-----------------

Adapting all seven layers (rather than just the attention layers) has been shown empirically to improve instruction-following performance, as the MLP layers encode factual knowledge whereas the attention layers encode syntactic and relational patterns. For medical fine-tuning, adapting the MLP is especially important to update clinical factual recall.

A.5 DATA PREPARATION PIPELINE

The data preparation stage consists of three sequential steps: (1) loading the medical reasoning dataset from HuggingFace Hub, (2) reformatting it into multi-turn conversation structures, and (3) applying the Qwen-3 chat template to produce the final training strings.

F.5.1 Chat Template Configuration

Qwen-3 uses the **ChatML** (Chat Markup Language) format, which wraps each conversational turn in special control tokens:

Qwen-3 ChatML Format — Control Token Structure

```
<|im_start|>system
You are Sehat AI, a medical assistant.<|im_end|>
<|im_start|>user
What are symptoms of acute appendicitis?<|im_end|>
<|im_start|>assistant
Acute appendicitis typically presents with...<|im_end|>
```

Chat Template Setup

```
from unsloth.chat_templates import get_chat_template

tokenizer = get_chat_template(
    tokenizer,
    chat_template = "qwen3-thinking",
)
```

The qwen3-thinking template variant also includes support for <think> / </think> delimiters for chain-of-thought reasoning traces. During inference, this can be toggled via enable_thinking=True/False.

A.5.2 Dataset Loading

Dataset Loading

```
from datasets import load_dataset

# Load custom medical reasoning dataset
dataset = load_dataset(
    "mabdulre9/medical-r1-distill-processed",
    split = "train"
)
```

Dataset Property	Details
HuggingFace Identifier	mabdulre9/medical-r1-distill-processed
Source	Curated subset of Medical-R1-Distill-Data with DeepSeek R1 CoT traces
Split Used	"train" (full training split)
Schema Fields	"question" (patient query), "answer" (physician response with reasoning)
Content Domain	Clinical Q&A covering symptom assessment, diagnosis, treatment options, medication guidance
Language	English (bilingual Urdu support added via vocabulary fine-tuning)

A.5.3 Conversation Format Conversion

The raw dataset rows (flat question/answer pairs) are restructured into HuggingFace's standard multi-turn conversation format — a list of role/content dictionaries. This format is required by the tokenizer's `apply_chat_template()` method.

Conversation Structuring

```
def generate_conversation(examples):
    problems = examples["question"]
    solutions = examples["answer"]
    conversations = []
    for problem, solution in zip(problems, solutions):
        conversations.append([
            {"role": "user", "content": problem},
            {"role": "assistant", "content": solution},
        ])
    return {"conversations": conversations}

dataset = dataset.map(generate_conversation, batched=True)
```

The `batched=True` argument to `dataset.map()` passes lists of examples rather than individual rows, which is significantly faster — HuggingFace processes multiple rows per Python call, reducing the per-row overhead from tokenisation and function call dispatch.

A.5.4 Chat Template Application

Applying Qwen-3 Chat Template

```
def formatting_prompts_func(examples):
    convos = examples["conversations"]
    texts = [
```

```

tokenizer.apply_chat_template(
    convo,
    tokenize      = False,
    add_generation_prompt = False
)
for convo in convos
]
return {"text": texts}

dataset = dataset.map(formatting_prompts_func, batched=True)

```

After this step, each example's text field contains the fully-formatted string that the model will train on. A representative example looks like:

Example Formatted Training String (with thinking trace)

```

<|im_start|>user
A 52-year-old male presents with sudden-onset crushing chest pain.
What is the most likely diagnosis?<|im_end|>
<|im_start|>assistant
<think>
The presentation is classic for STEMI: sudden crushing chest pain in a middle-aged
male...
</think>
The most likely diagnosis is ST-Elevation Myocardial Infarction
(STEMI)...<|im_end|>

```

Training is implemented using HuggingFace TRL's SFTTrainer, which is a wrapper around Trainer that adds: (1) automatic packing of short sequences into fixed-length blocks to maximise GPU utilisation, and (2) integration with the `train_on_responses_only` masking utility.

SFTTrainer Configuration

```
from trl import SFTTrainer, SFTConfig

trainer = SFTTrainer(
    model      = model,
    tokenizer   = tokenizer,
    train_dataset = dataset,
    eval_dataset = None,
    args = SFTConfig(
        dataset_text_field      = "text",
        per_device_train_batch_size = 2,
        gradient_accumulation_steps = 4,
        warmup_steps             = 5,
        num_train_epochs          = 3,
        learning_rate             = 2e-4,
        logging_steps             = 1,
        optim                     = "adamw_8bit",
        weight_decay              = 0.001,
        lr_scheduler_type         = "linear",
        seed                      = 3407,
        report_to                 = "none",
    ),
)
```

A.6.1 Hyperparameter Reference Table

Hyperparameter	Value & Technical Rationale
per_device_train_batch_size	2. The maximum batch that fits in VRAM at max_seq_len=2048 with 4-bit model + LoRA gradients. Increasing to 4 causes OOM errors.
gradient_accumulation_steps	4. Simulates an effective batch size of $2 \times 4 = 8$. Gradients are accumulated over 4 micro-batches before the optimiser step, without additional VRAM cost.
warmup_steps	5. Linearly increases the learning rate from 0 to learning_rate over 5 optimiser steps. Prevents large gradient updates at the start of training when the LoRA matrices are far from optimal.
num_train_epochs	3. Three complete passes through the training dataset. For a medical Q&A dataset of several thousand examples, 3 epochs typically achieves convergence without overfitting.
learning_rate	$2e-4$ (0.0002). Standard choice for LoRA fine-tuning. Significantly higher than full fine-tuning rates ($2e-5$) because only the small LoRA matrices are updated — they can absorb larger updates safely.
logging_steps	1. Logs loss after every optimiser step, allowing fine-grained monitoring of the loss curve.
optim	"adamw_8bit". 8-bit AdamW stores optimiser moments in INT8 rather than FP32, saving ~75% of the memory that would otherwise be consumed by Adam's first and second

	moment estimates.
weight_decay	0.001. L2 regularisation coefficient. Small value to prevent adapter weight explosion without suppressing useful adaptation signal.
lr_scheduler_type	"linear". The learning rate decays linearly from learning_rate to 0 over the training duration after the warmup phase.
seed	3407. Global random seed for reproducible shuffling of the dataset and model initialisation.
report_to	"none". Disables W&B / TensorBoard logging.

A.6.2 Response-Only Training Mask

A critical technique applied in this notebook is **training on responses only** — the loss is computed exclusively on the assistant's output tokens, while user prompt tokens are masked to -100 (the PyTorch ignore index for CrossEntropyLoss).

Train_on_responses_only Masking

```
from unsloth.chat_templates import train_on_responses_only

# Only compute loss on assistant outputs, not user prompts
trainer = train_on_responses_only(
    trainer,
    instruction_part = "<|im_start|>user\n",
    response_part    = "<|im_start|>assistant\n",
)
```

Without this masking, the model would also receive gradient signal from the user's question tokens — teaching it to predict how users phrase questions (which is not useful) rather than how to generate high-quality medical answers. The masking provides roughly a **1.5–2× improvement** in effective task-specific learning per training step.

⚙ **Note:** *Masking is verified by decoding the "labels" tensor for a sample: the user prompt region appears as padding tokens while only the assistant response tokens are visible.*

The notebook includes an explicit GPU memory profiling routine to measure the memory footprint before and after training. This is essential for understanding the cost distribution between the quantised base model weights, the LoRA adapter activations, and the 8-bit AdamW optimiser states.

GPU Profiling and Training Launch

```
# Show pre-training GPU memory stats
gpu_stats = torch.cuda.get_device_properties(0)
start_gpu_memory = round(
    torch.cuda.max_memory_reserved() / 1024 / 1024 / 1024, 3
)
max_memory = round(gpu_stats.total_memory / 1024 / 1024 / 1024, 3)
print(f'GPU = {gpu_stats.name}. Max memory = {max_memory} GB.')
print(f'{start_gpu_memory} GB of memory reserved.')

# Run training
trainer_stats = trainer.train()

# Show post-training stats
used_memory = round(torch.cuda.max_memory_reserved() / 1024**3, 3)
used_memory_for_lora = round(used_memory - start_gpu_memory, 3)
used_pct = round(used_memory / max_memory * 100, 3)
lora_pct = round(used_memory_for_lora / max_memory * 100, 3)
print(f'Train runtime : {trainer_stats.metrics['train_runtime']:.1f} s')
print(f'Peak memory : {used_memory} GB ({used_pct}% of GPU)')
print(f'LoRA overhead : {used_memory_for_lora} GB ({lora_pct}% of GPU)')
```

A.7.1 Expected Memory Breakdown

Memory Component	Approximate Size
Base model weights (NF4 4-bit)	~2.6 GB
LoRA adapter weights (fp32)	~0.08 GB (r=32, all 7 layers)
Activation memory (gradient checkpointing)	~0.5–1.0 GB (recomputed, not stored)
8-bit AdamW optimiser states	~0.16 GB (2 × LoRA params, INT8)
Input batch tokens (batch=2, seq=2048)	~0.03 GB
Total Peak Reserved (typical)	~4.5–6.0 GB of 16 GB

A.7.2 Interpreting Training Statistics

After `trainer.train()` completes, `trainer_stats.metrics` contains the following key fields:

- **train_runtime:** Total wall-clock seconds for training. Divide by 60 for minutes.
- **train_samples_per_second:** Throughput measure — higher is better.
- **train_loss:** Final average cross-entropy loss on assistant tokens.
- **epoch:** Fractional epoch count if `max_steps` was set instead of `num_epochs`.

⚙ **Note:** For a 3-epoch run on the *medical-r1-distill* dataset (~5,000 samples), typical final loss values range from 0.8–1.4 for a well-configured medical LoRA. Loss below 0.5 may indicate overfitting.

Once trained, the model can be used for inference using HuggingFace's standard `model.generate()` API. The notebook demonstrates inference with the `TextStreamer`, which prints tokens to `stdout` as they are generated rather than waiting for the entire response.

Inference with Streaming Output

```
messages = [
    {"role": "user",
     "content": "A 45-year-old presents with chest pain radiating to the left arm."},
]

text = tokenizer.apply_chat_template(
    messages,
    tokenize      = False,
    add_generation_prompt = True,
    enable_thinking = False, # disable chain-of-thought mode
)

from transformers import TextStreamer

_ = model.generate(
    **tokenizer(text, return_tensors="pt").to("cuda"),
    max_new_tokens = 450,
    temperature    = 0.7,
    top_p          = 0.8,
    top_k          = 20,
    streamer       = TextStreamer(tokenizer, skip_prompt=False),
)
```

A.8.1 Inference Parameter Reference

Parameter	Value & Effect
max_new_tokens	450. Maximum tokens to generate after the prompt. For medical responses, 450 tokens is sufficient for a detailed clinical answer. Increase to 1024+ for longer differential diagnosis outputs.
temperature	0.7. Controls output randomness. At 0.7, the model produces varied but coherent responses. Reduce to 0.1–0.3 for more deterministic clinical answers (e.g., drug dosage queries).
top_p	0.8. Nucleus sampling — at each step, the model samples only from the smallest set of tokens whose cumulative probability ≥ 0.8 . Prevents very low-probability tokens from being selected.
top_k	20. At each step, only the top-20 most probable tokens are candidates for sampling. Reduces incoherence by excluding tail-probability tokens.
enable_thinking	False. Disables the Qwen-3 chain-of-thought reasoning mode. Set to True to get <think>...</think> reasoning traces in the output — useful for diagnostic explanation.
TextStreamer	Streams tokens to stdout as generated, providing real-time output without buffering the full response.

A.8.2 Thinking vs. Non-Thinking Inference

The Qwen-3 Thinking architecture supports two inference modes:

- **Non-thinking mode** (`enable_thinking=False`): `temperature=0.7`, `top_p=0.8`, `top_k=20`. Recommended for production clinical use — fast, focused, no visible reasoning trace.
- **Thinking mode** (`enable_thinking=True`): `temperature=0.6`, `top_p=0.95`, `top_k=20`. Generates a `<think>` block containing step-by-step clinical reasoning before the final answer. Useful for explainability audits.

⚙️ **Note:** In the Sehat AI Flask deployment, `enable_thinking` is set to `False` and `temperature` is reduced to `0.2` via the Ollama Modelfile to maximise clinical accuracy and response consistency.

A.9 SAVING AND EXPORTING THE FINE-TUNED MODEL

The notebook provides three export pathways depending on the deployment target: (1) saving only the LoRA adapters for adapter-based loading, (2) merging adapters with base weights for standard deployment, and (3) quantising to GGUF for Ollama/llama.cpp offline inference.

A.9.1 *Saving LoRA Adapters Only*

This saves only the **A** and **B** matrices for each adapted layer — approximately **80–150 MB** rather than the full ~8 GB fp16 model. This is sufficient for deployment in environments that can load the base model separately.

Saving LoRA Adapter Weights

```
# Save LoRA adapter weights only (not full model)
model.save_pretrained("qwen_lora")
tokenizer.save_pretrained("qwen_lora")

# Push to HuggingFace Hub (optional)
model.push_to_hub("mabdulre9/qwen_lowrank", token="HF_TOKEN")
tokenizer.push_to_hub("mabdulre9/qwen_lowrank", token="HF_TOKEN")
```

The adapter directory contains: adapter_config.json (LoRA configuration), adapter_model.safetensors (weight matrices), and tokenizer_config.json + tokenizer.json (tokenizer files).

A.9.2 Merged Weight Export (Float16 and 4-bit)

For deployment scenarios where the base model is not separately available — such as the Sehat AI Flask server — the LoRA adapters can be merged into the base weights and saved as a self-contained model.

Merged Weight Export

```
# Merge weights + save as float16 (for vLLM deployment)
model.save_pretrained_merged(
    "qwen_finetune_16bit", tokenizer,
    save_method = "merged_16bit"
)

# Merge weights + save as 4-bit int (compact deployment)
model.push_to_hub_merged(
    "mabdulre9/qwen_finetune_4bit", tokenizer,
    save_method = "merged_4bit",
    token = "HF_TOKEN"
)
```

Export Format	Use Case & Trade-offs
merged_16bit	Full fp16 model (~8 GB). Best accuracy. Use for vLLM or high-VRAM server deployment.
merged_4bit	Quantised 4-bit model (~2.6 GB). Slight accuracy reduction. Best for resource-constrained deployment.
LoRA adapters only	~80–150 MB. Requires base model at inference time. Best for development / iterative experiments.

A.9.3 GGUF Export for Ollama / llama.cpp

The final export step converts the merged weights to GGUF (GPT-Generated Unified Format), the file format used by llama.cpp and Ollama for cross-platform CPU/GPU inference. This is the format used in the Sehat AI offline deployment.

GGUF Quantisation and Export

```
# Export as Q4_K_M GGUF (recommended for Ollama/llama.cpp)
model.push_to_hub_gguf(
    "mabdulre9/qwen_finetune",
    tokenizer,
    quantization_method = "q4_k_m",
    token = "HF_TOKEN"
)

# Export multiple GGUF quant levels simultaneously
model.push_to_hub_gguf(
    "mabdulre9/qwen_finetune",
    tokenizer,
    quantization_method = ["q4_k_m", "q8_0", "q5_k_m"],
    token = "HF_TOKEN"
)
```

GGUF Quantisation Method	Description and Quality/Size Trade-off
q8_0	8-bit uniform quantisation. Near-lossless quality (~99.5% of fp16). ~4.2 GB. Recommended for development.
q4_k_m	4-bit with mixed K-quantisation. Uses Q6_K for half of

	critical attention/MLP layers, Q4_K elsewhere. ~2.4 GB. Best accuracy/size balance. USED IN SEHAT AI.
q5_k_m	5-bit K-quantisation variant. ~3.0 GB. Slightly higher quality than q4_k_m with modest size increase.
f16	Full float16 GGUF. ~8 GB. No quality loss, but large file size.

⚙️ **Note:** The Sehat AI deployment uses q4_k_m GGUF (2.6 GB) imported into Ollama via a custom Modelfile. The Q4_K_M format specifically preserves quality in the attention.wv and feed_forward.w2 tensors using Q6_K, which are the most sensitive to quantisation error.

A.10 RELOADING SAVED ADAPTERS FOR INFERENCE

The notebook includes a guard-block (disabled by default with `if False:`) demonstrating how to reload the saved LoRA adapters for inference without re-running the training pipeline:

Reloading Saved LoRA Adapters

```
from unsloth import FastLanguageModel

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "qwen_lora", # path to saved adapter
    max_seq_length = 2048,
    load_in_4bit = True,
)
```

When loading adapters via Unsloth's `FastLanguageModel.from_pretrained()`, the adapter configuration is automatically read from `adapter_config.json` in the model directory. Unsloth patches the model with LoRA layers and loads the saved A/B matrices in a single call — no separate `PeftModel.from_pretrained()` call is needed.

A.10.1 Deployment Architecture Summary

Component	Technology / Config
Training Framework	Unsloth + HuggingFace TRL SFTTrainer
Base Model	Qwen3-4B-Thinking-2507 (NF4 4-bit load)
Adapter Method	LoRA, $r=32$, $\alpha=32$, all 7 transformer linear layers

Dataset	mabdulre9/medical-r1-distill-processed (Hugging Face)
Chat Format	Qwen-3 ChatML (qwen3-thinking template)
Training Objective	Causal LM loss on assistant tokens only (response masking)
Optimiser	AdamW 8-bit, lr=2e-4, linear decay, warmup=5 steps
Effective Batch Size	8 (per_device=2 × grad_accumulation=4)
Epochs	3 full epochs over training dataset
Final Export Format	Q4_K_M GGUF via llama.cpp (2.6 GB)
Inference Runtime	Ollama with custom Modelfile (temperature=0.2)
Deployment Backend	Python Flask + Ollama REST API

A.11 END-TO-END PIPELINE SUMMARY

The following flow captures the complete trajectory from environment setup to clinical deployment as implemented in `finetuning.ipynb`:

1. **Install dependencies:** Conditional local pip installation with pinned library versions.
2. **Load base model:** Qwen3-4B-Thinking-2507 in 4-bit NF4 via Unsloth `FastLanguageModel`.
3. **Inject LoRA adapters:** $r=32$, $\alpha=32$, applied to q/k/v/o projections and gate/up/down MLP layers.
4. **Configure chat template:** Apply qwen3-thinking ChatML tokeniser template.
5. **Load dataset:** medical-r1-distill-processed from HuggingFace Hub.
6. **Format conversations:** Map question/answer pairs to `[{role, content}]` conversation objects.
7. **Apply chat template:** Render conversations to full training strings with control tokens.
8. **Configure SFTTrainer:** `batch=2`, `grad_accum=4`, `lr=2e-4`, 3 epochs, AdamW-8bit.
9. **Apply response masking:** Mask user prompt tokens; compute loss only on assistant tokens.
10. **Profile GPU memory:** Record baseline and peak VRAM usage.
11. **Execute training:** `trainer.train()` — runs full supervised fine-tuning loop.
12. **Test inference:** Validate model with TextStreamer streaming output.
13. **Save LoRA adapters:** Local save + HuggingFace Hub push.
14. **Export merged model:** Merge adapters into base weights $\rightarrow$ fp16 and 4-bit variants.
15. **GGUF conversion:** Convert to Q4_K_M GGUF for Ollama deployment.

16. **Ollama integration:** Import GGUF via Modelfile → serve via REST API → Flask backend.

— *End of Annexure A* —

One page summary of plagiarism report signed by the supervisor

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

-  **65 Not Cited or Quoted 7%**
Matches with neither in-text citation nor quotation marks
-  **4 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 5%  Internet sources
- 3%  Publications
- 2%  Submitted works (Student Papers)